

Computer Vision – Lecture 11

CNNs IV

26.05.2025

Bastian Leibe

Visual Computing Institute

RWTH Aachen University

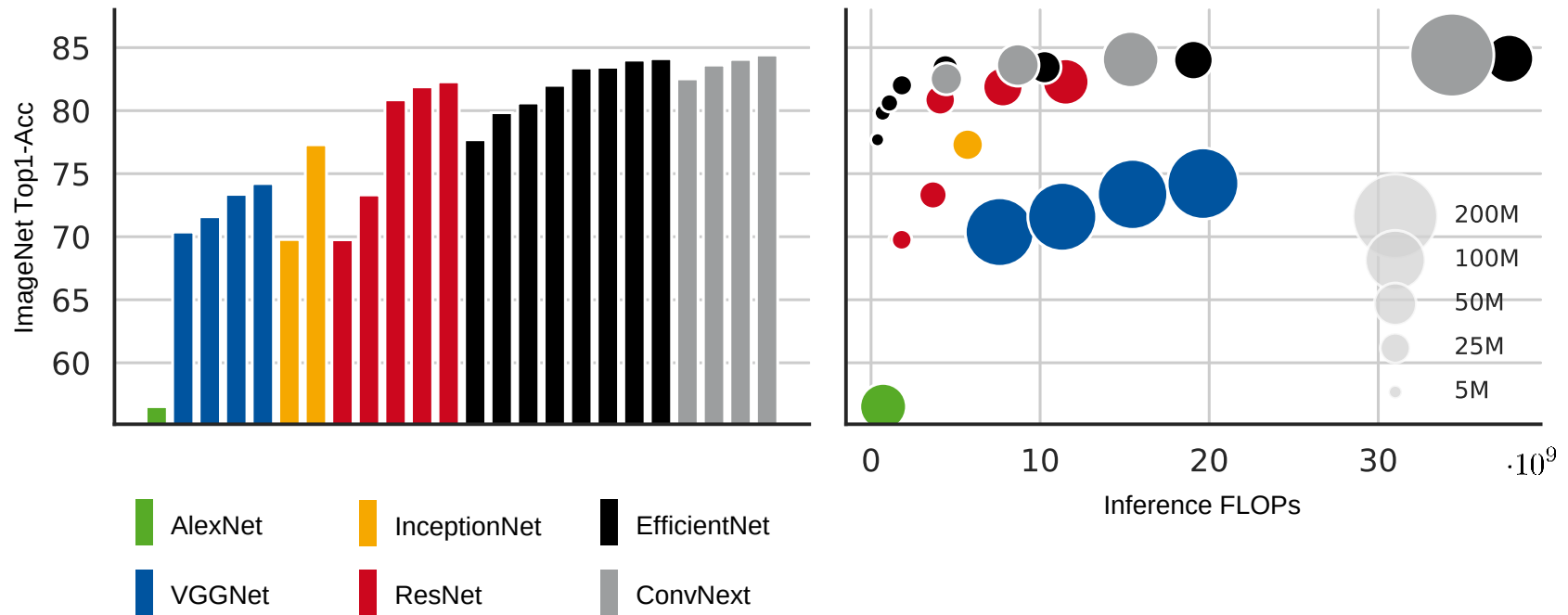
<http://www.vision.rwth-aachen.de/>

leibe@vision.rwth-aachen.de

Course Outline

- Image Processing Basics
- Segmentation & Grouping
- Object Recognition & Categorization
 - Sliding Window based Object Detection
- Deep Learning for Vision
 - Convolutional Neural Networks (CNNs)
 - Deep Learning Background
 - CNNs for Object Detection
 - CNNs for Semantic Segmentation
 - CNNs for Matching
 - Transformers
- 3D Reconstruction

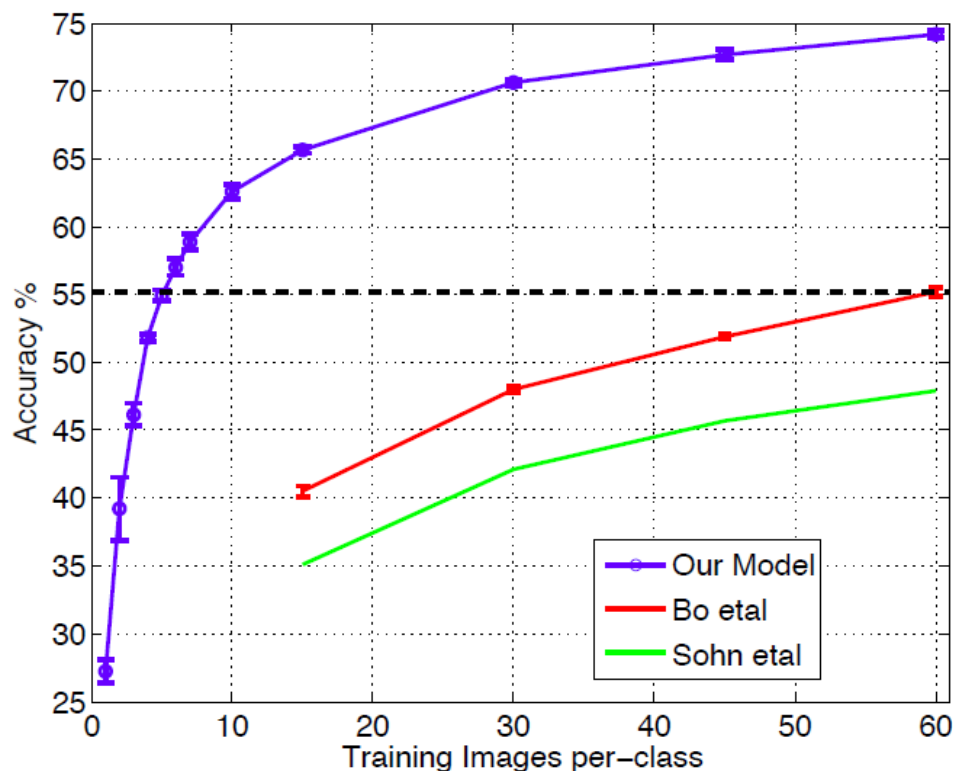
Recap: CNN Backbone Architectures



Models trained on ImageNet-1K, as available in torchvision v0.17.2

• g

The Learned Features are Generic



state of the art
level (pre-CNN)

- Experiment: feature transfer
 - Train AlexNet-like network on ImageNet
 - Chop off last layer and train classification layer on CalTech256
- ⇒ State of the art accuracy already with only 6 training images!

Transfer Learning with CNNs

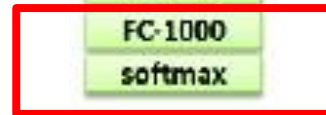


1. Train on
ImageNet



2. If small dataset: fix all
weights (treat CNN as
fixed feature extrac-
tor), retrain only the
classifier

I.e., swap the Softmax
layer at the end



Transfer Learning with CNNs



1. Train on
ImageNet



3. If you have medium
sized dataset,
“[finetune](#)” instead: use
the old weights as
initialization, train the
full network or only
some of the higher
layers.

Retrain bigger portion
of the network

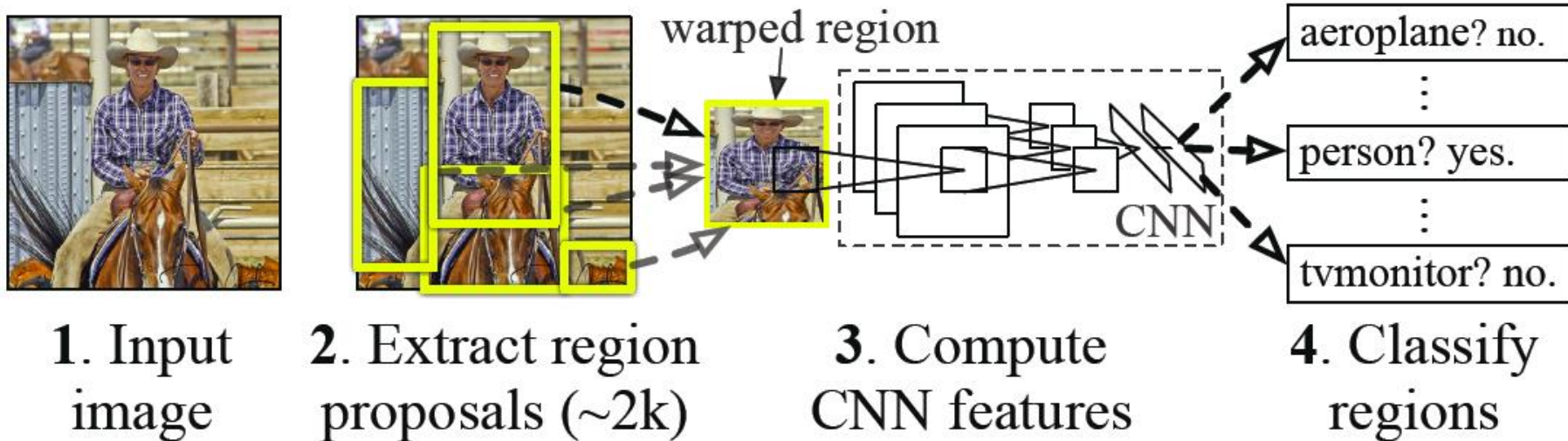


Topics of This Lecture

- CNNs for Object Detection
 - The R-CNN Family
 - The YOLO Family
- CNNs for Segmentation
 - Fully Convolutional Networks (FCN)
 - Encoder-Decoder architecture
 - Transpose convolutions
 - Skip connections
- CNNs for Human Pose Estimation
 - 2D Body pose estimation
 - 3D Body pose estimation
- CNNs for Matching
 - Siamese networks
 - Triplet loss

Object Detection: R-CNN Idea

R-CNN: *Regions with CNN features*

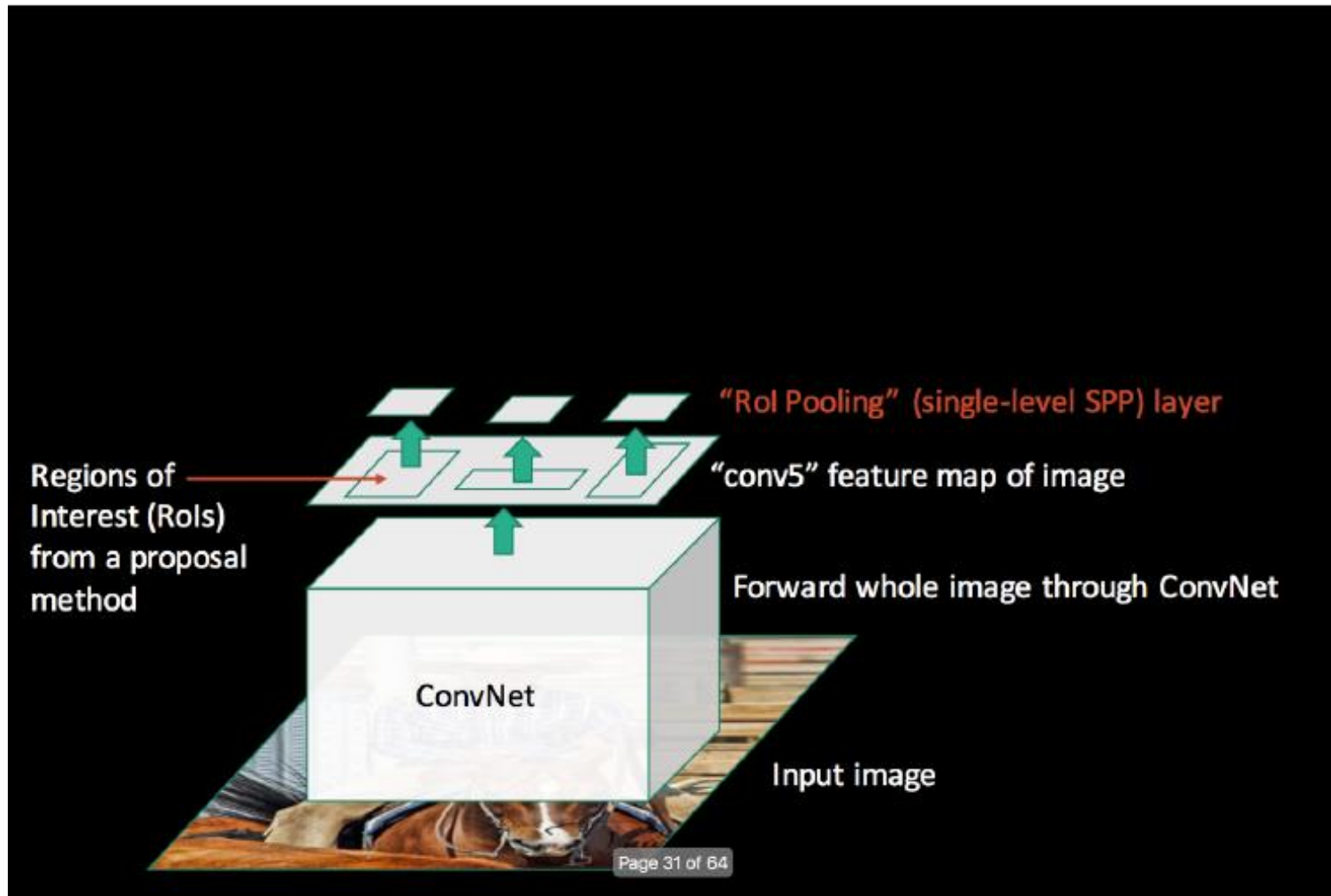


- Results on PASCAL VOC Detection benchmark
 - Pre-CNN state of the art: 35.1% mAP Selective Search
33.4% mAP DPM
 - R-CNN: 53.7% mAP

R. Girshick, J. Donahue, T. Darrell, and J. Malik, [Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation](#), CVPR 2014

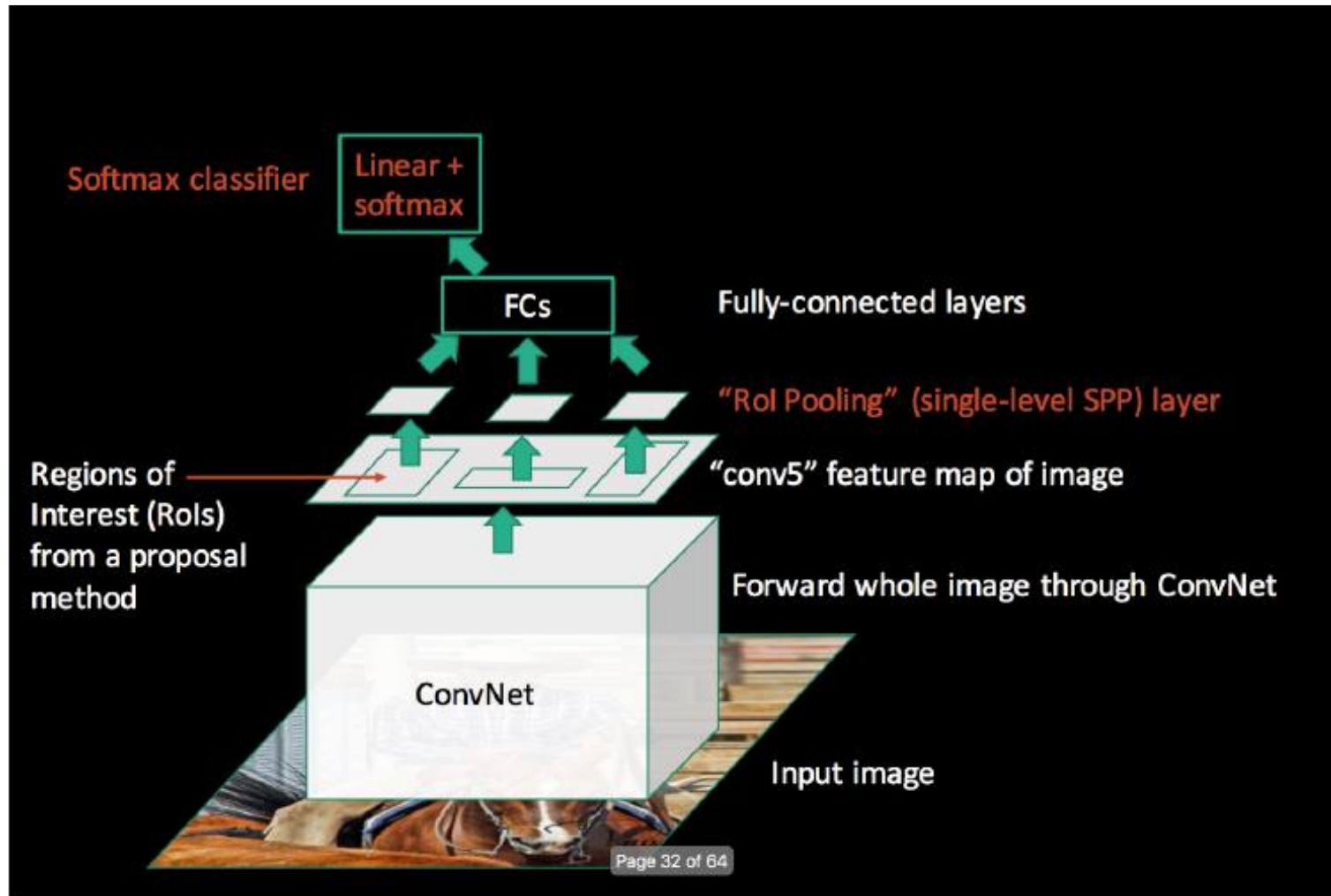
Improved Version: Fast R-CNN

- Forward Pass



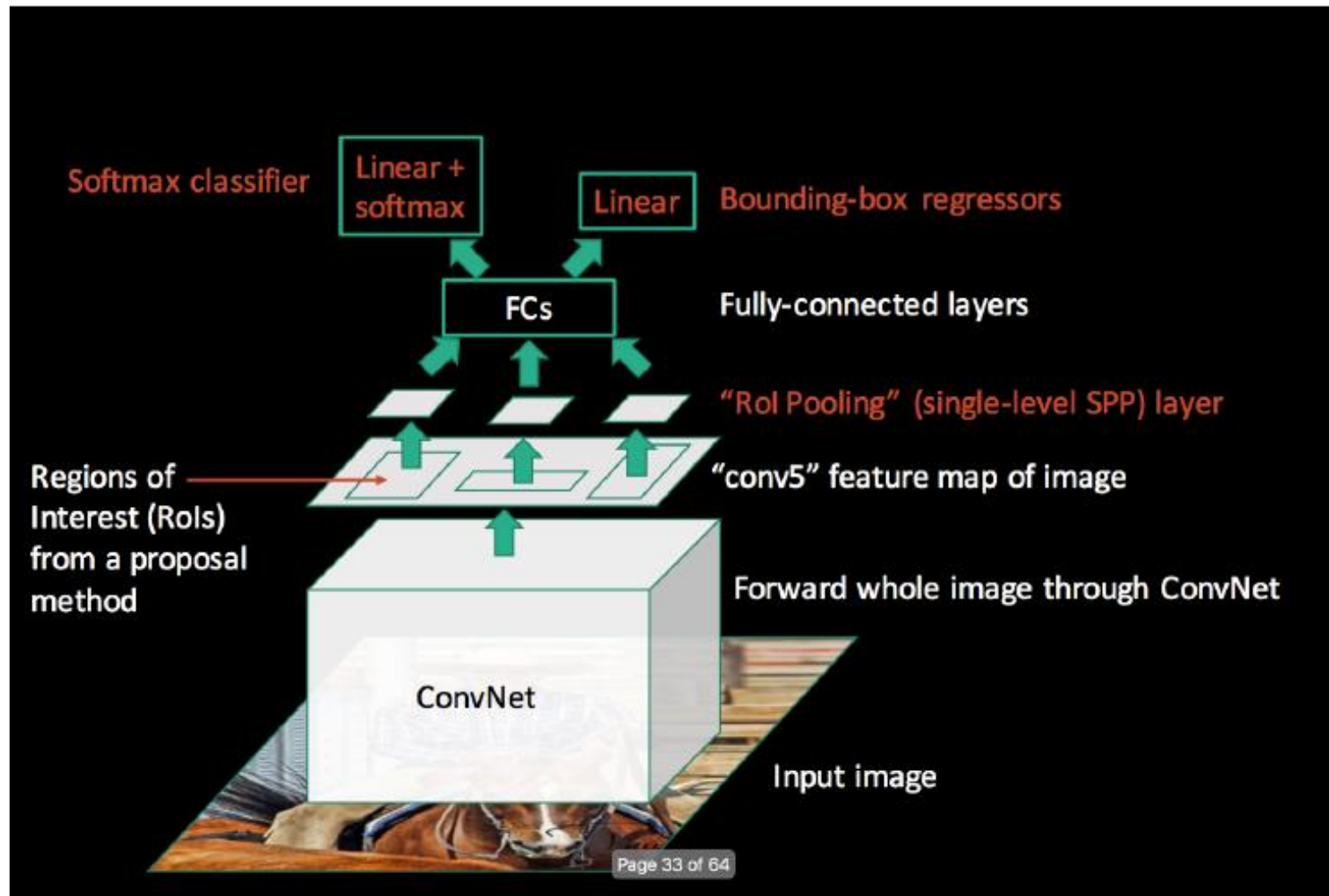
Fast R-CNN

- Forward Pass



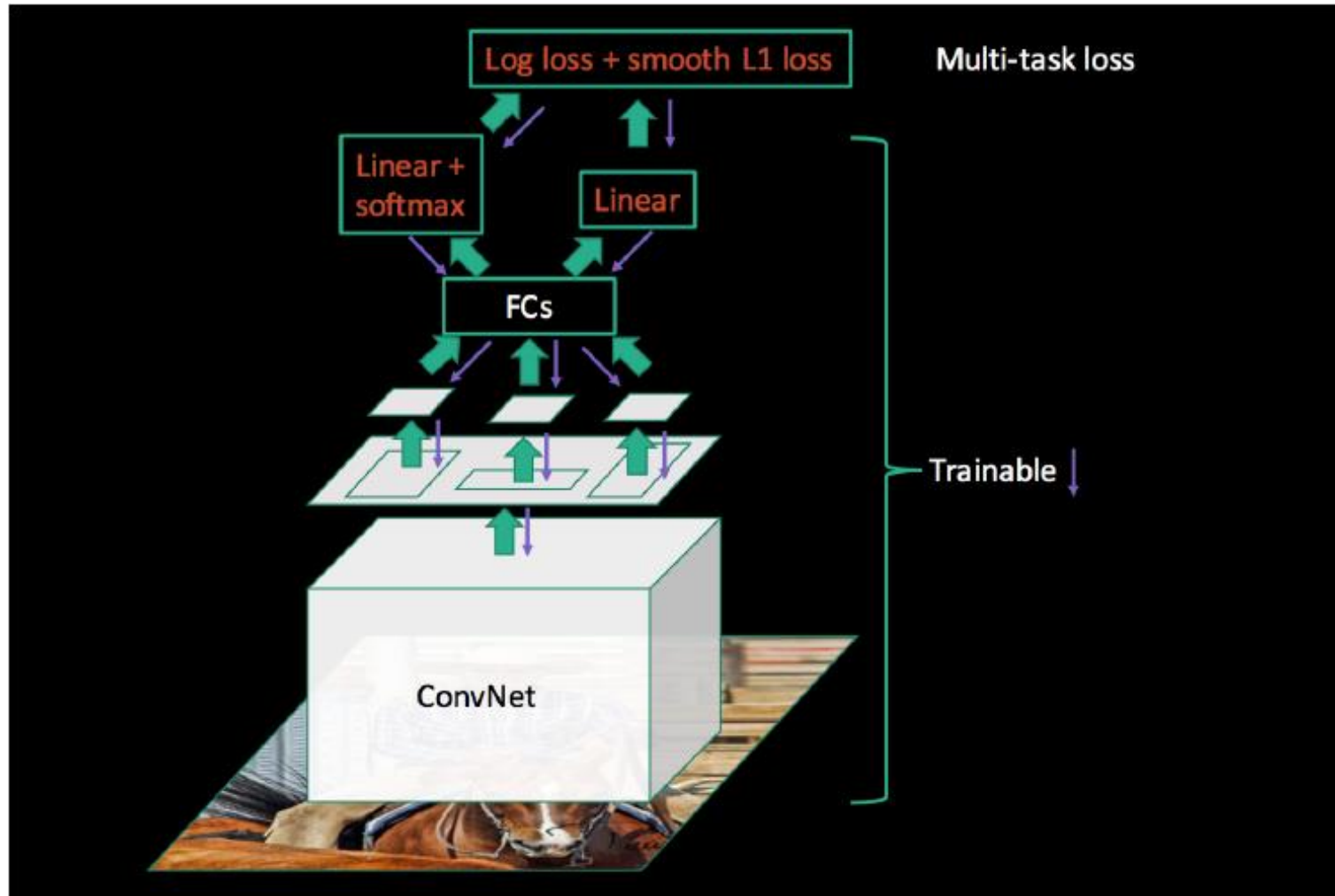
Fast R-CNN

- Forward Pass



Fast R-CNN Training

- Backward Pass

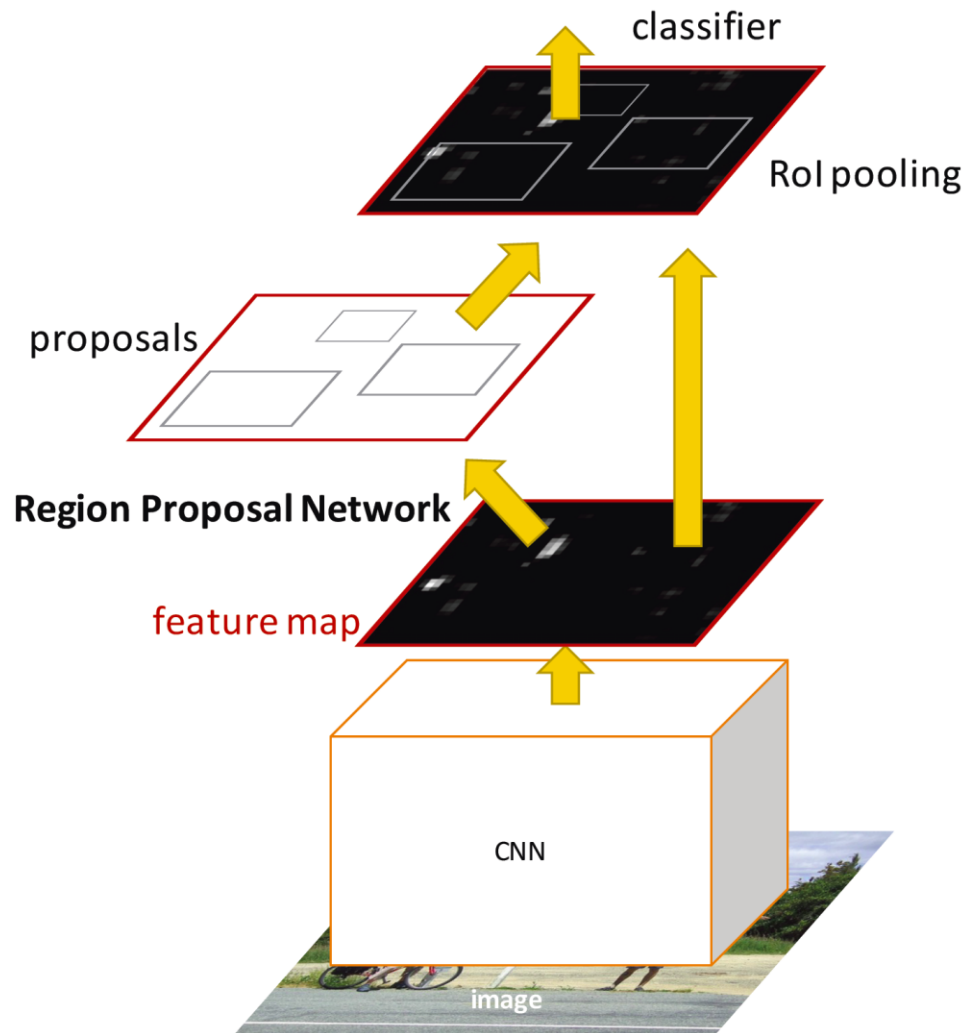


Region Proposal Networks (RPN)

- Idea
 - Remove dependence on external region proposal algorithm.
 - Instead, infer region proposals from same CNN.

⇒ Feature sharing

⇒ Object detection in a single pass becomes possible.
- Faster R-CNN = Fast R-CNN + RPN

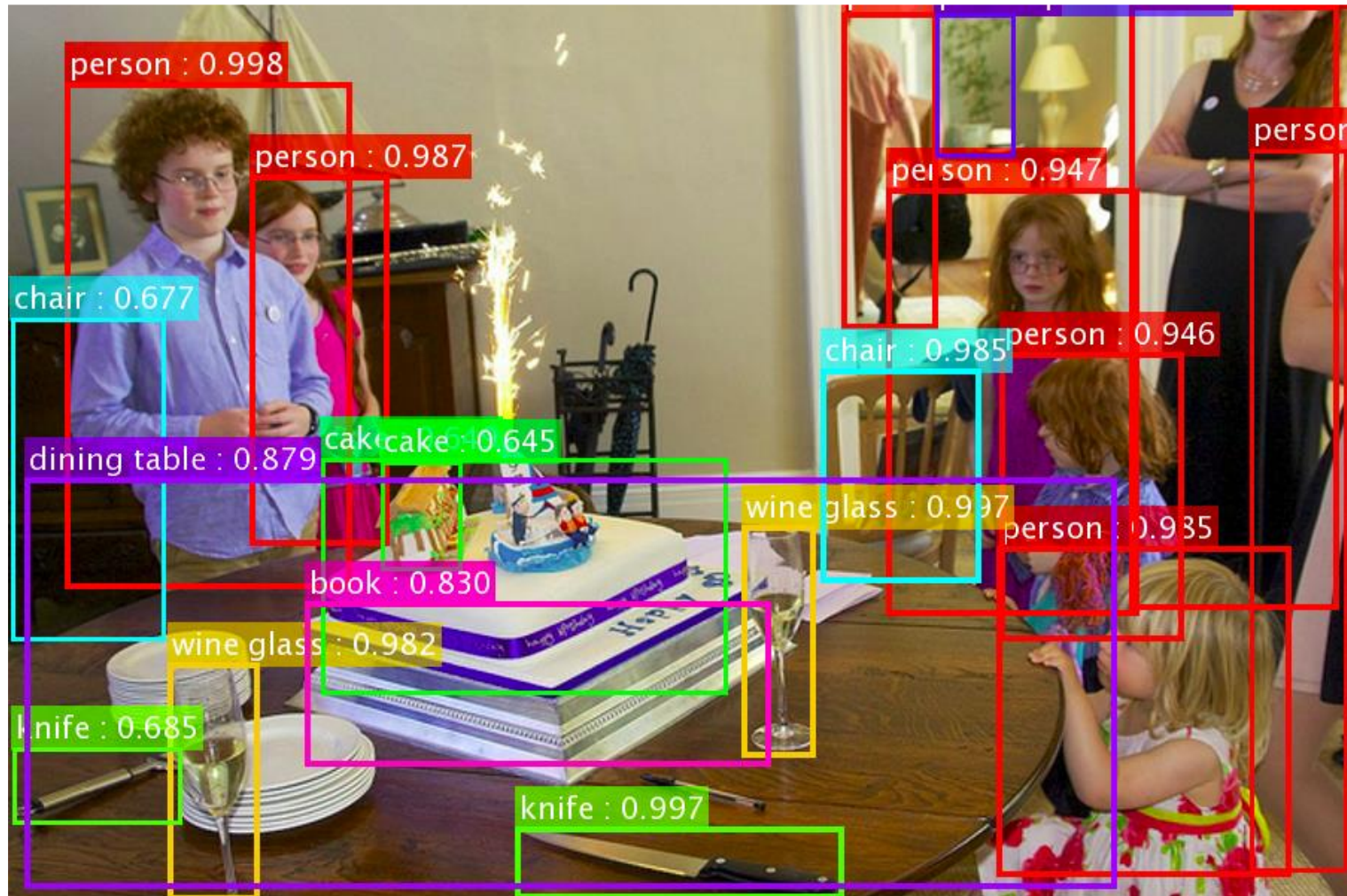


Faster R-CNN

- One network, four losses
 - Joint training

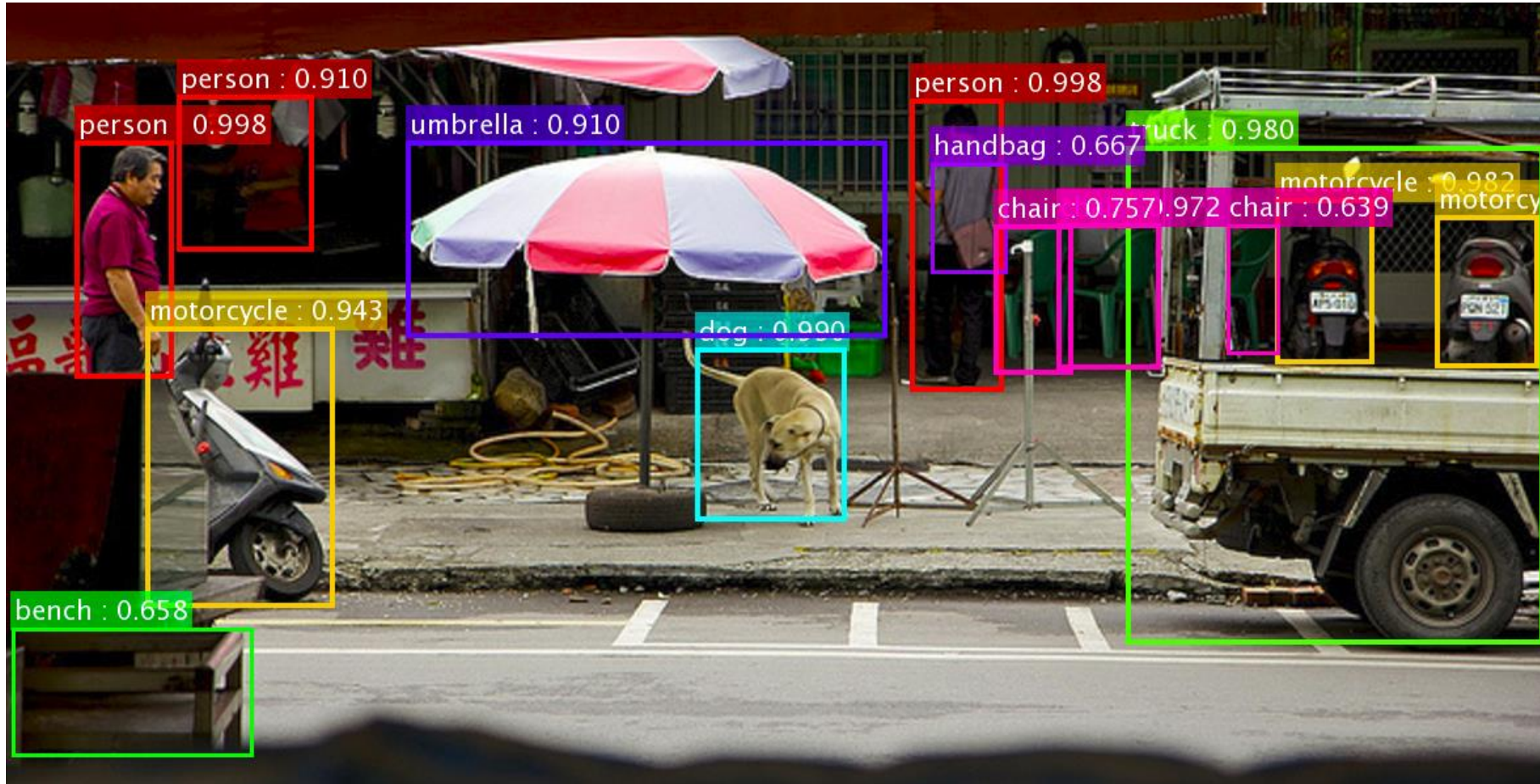


Faster R-CNN (based on ResNets)



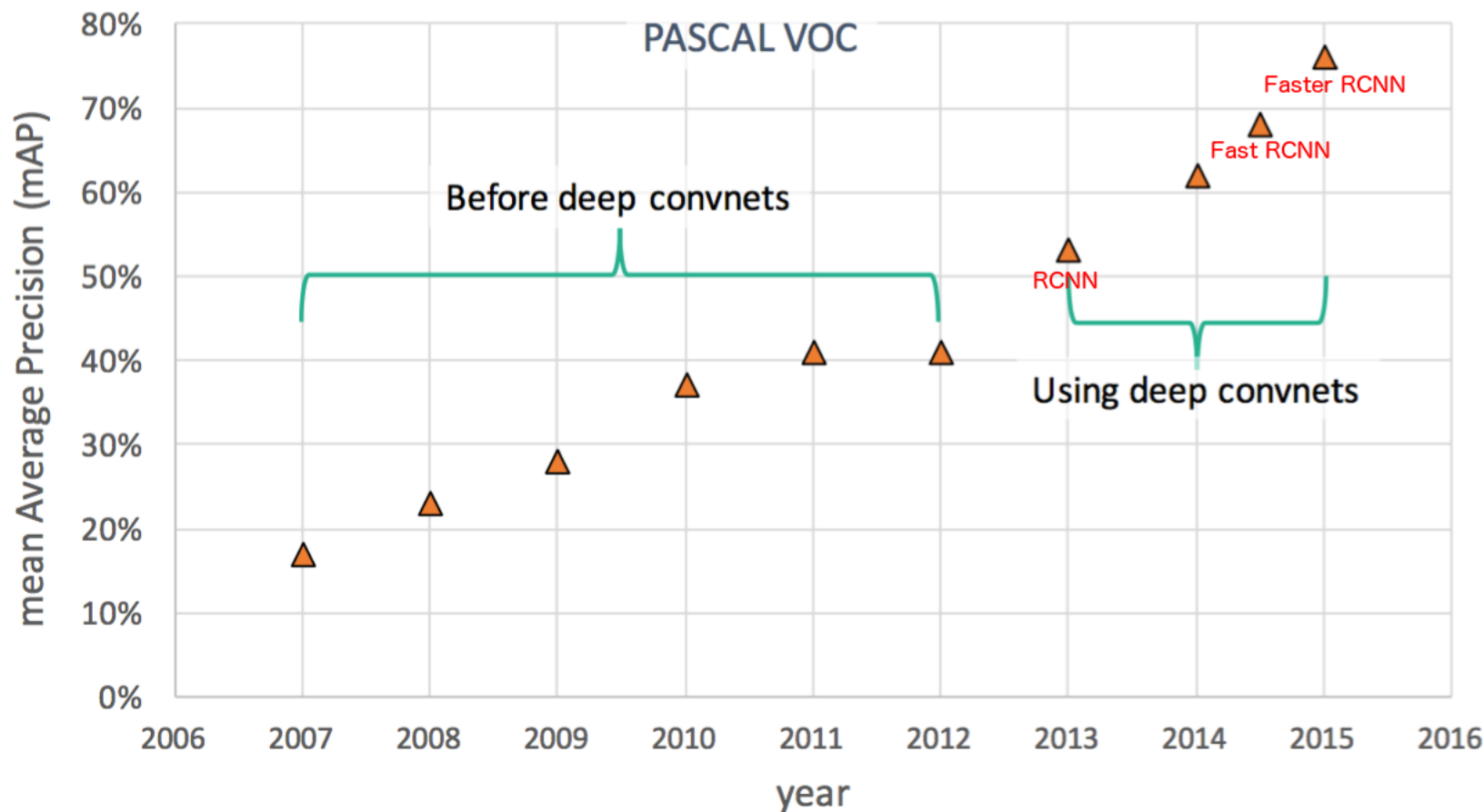
K. He, X. Zhang, S. Ren, J. Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016.

Faster R-CNN (based on ResNets)

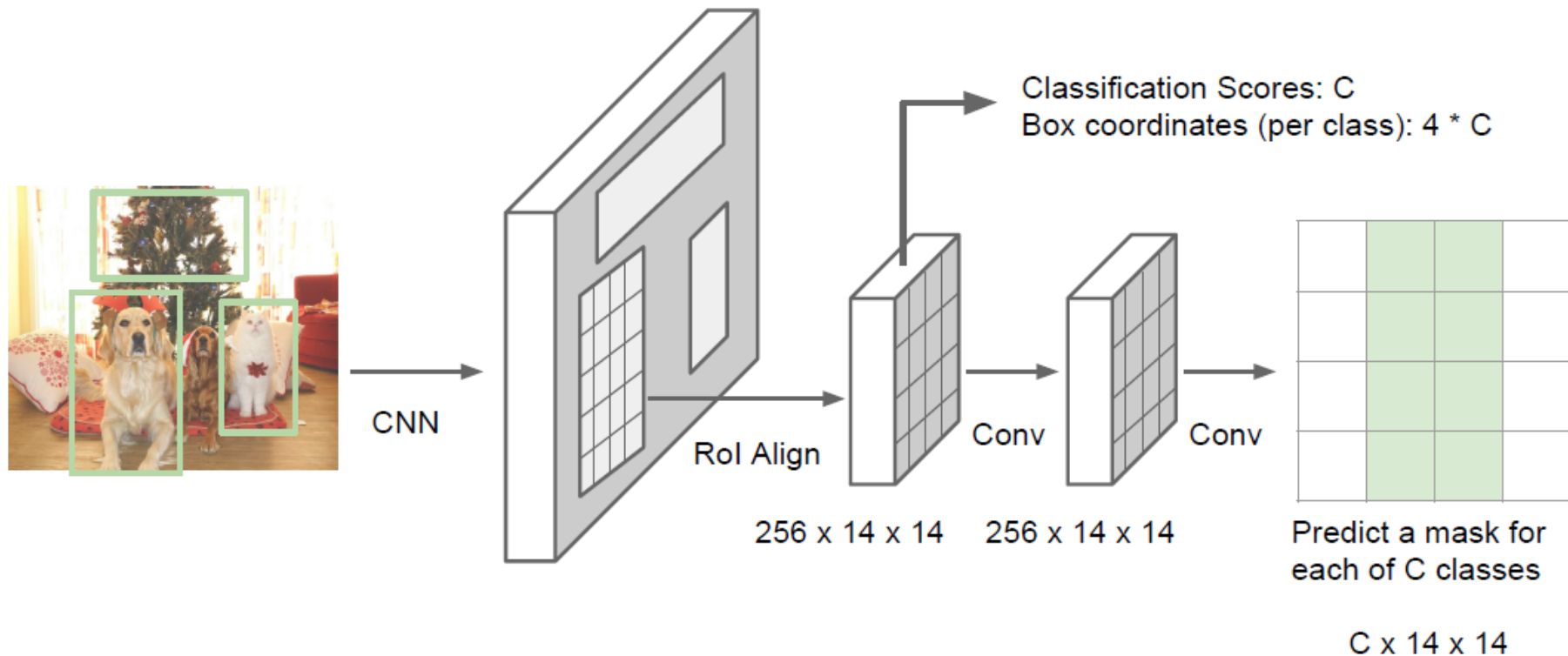


K. He, X. Zhang, S. Ren, J. Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016.

Object Detection Performance



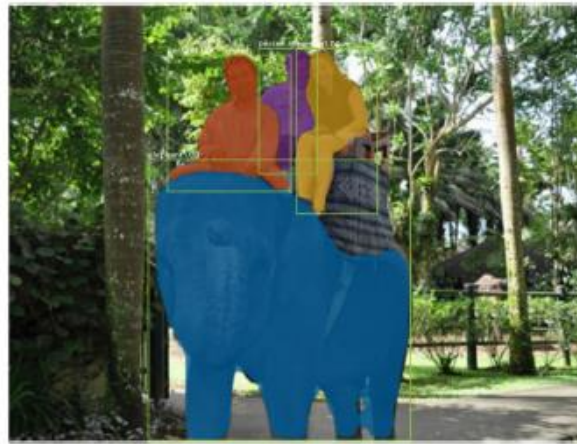
More Recent Version: Mask R-CNN



K. He, G. Gkioxari, P. Dollar, R. Girshick, [Mask R-CNN](#), arXiv 1703.06870.

Mask R-CNN Results

- Detection + Instance segmentation



- Detection + Pose estimation

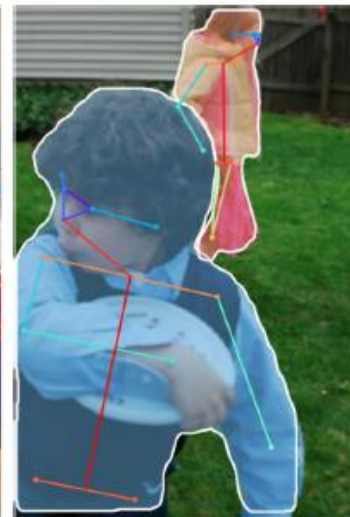
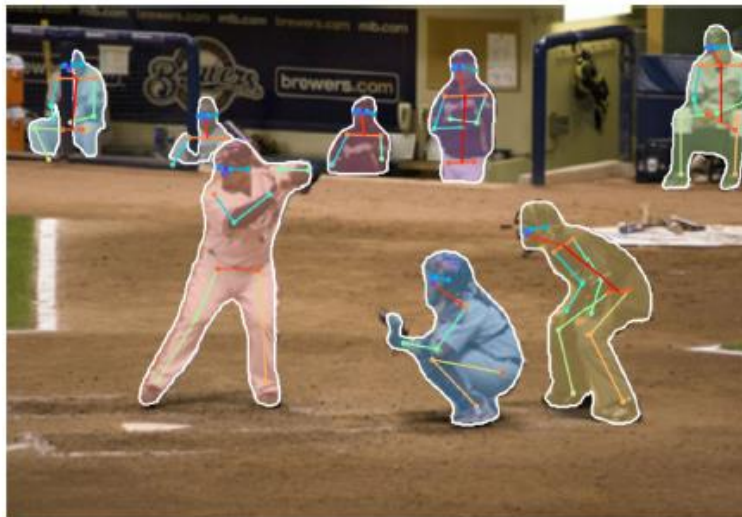
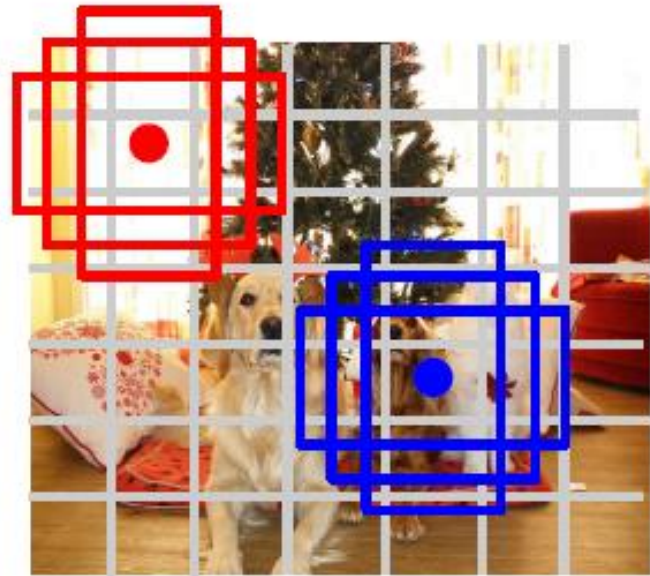


Figure credit: K. He, G. Gkioxari, P. Dollar, R. Girshick

YOLO / SSD



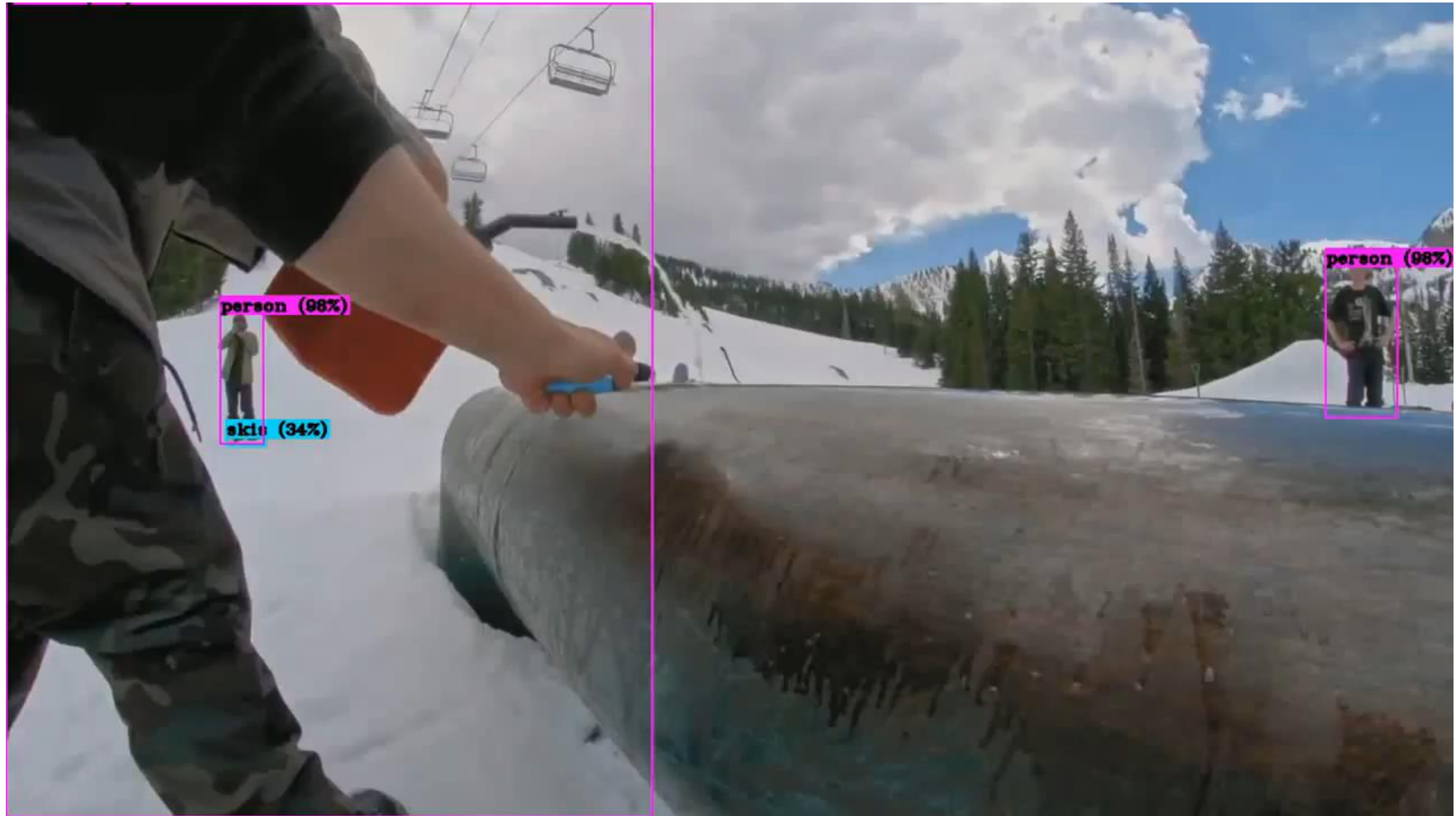
Input image
 $3 \times H \times W$



Divide image into grid
 7×7

- Idea: Directly go from image to detection scores
- Within each grid cell
 - Start from a set of anchor boxes
 - Regress from each of the B anchor boxes to a final box
 - Predict scores for each of C classes (including background)

YOLO-v4 Results



J. Redmon, S. Divvala, R. Girshick, A. Farhadi, [You Only Look Once: Unified, Real-Time Object Detection](#), CVPR 2016.

Summary

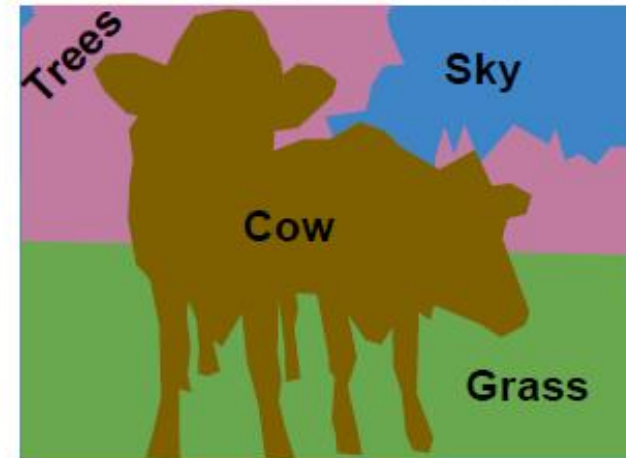
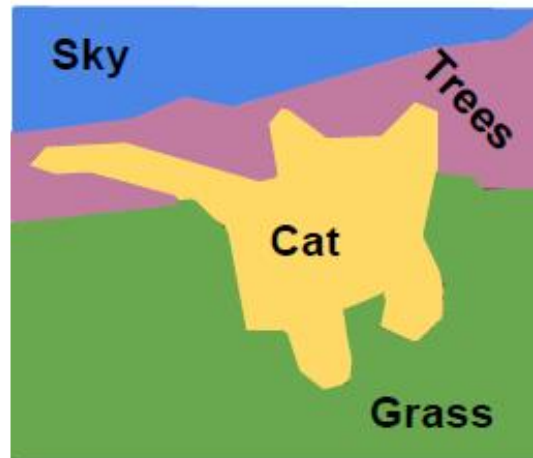
- Object Detection
 - Find a variable number of objects by classifying image regions
 - Before CNNs: dense multiscale sliding window (HoG, DPM)
- Region proposal based detectors
 - Idea: Avoid dense sliding window with region proposals
 - R-CNN: Selective Search + CNN classification / regression
 - Fast R-CNN: Swap order of convolutions and region extraction
 - Faster R-CNN: Compute region proposals within the network
 - **Mask R-CNN**: Detection + instance segmentation + pose estimation
- Anchor box based detectors
 - Idea: Perform detection in a single step using grid of anchor boxes
 - **YOLO**, YOLO-v2, YOLO-v3, ...
 - SSD

Topics of This Lecture

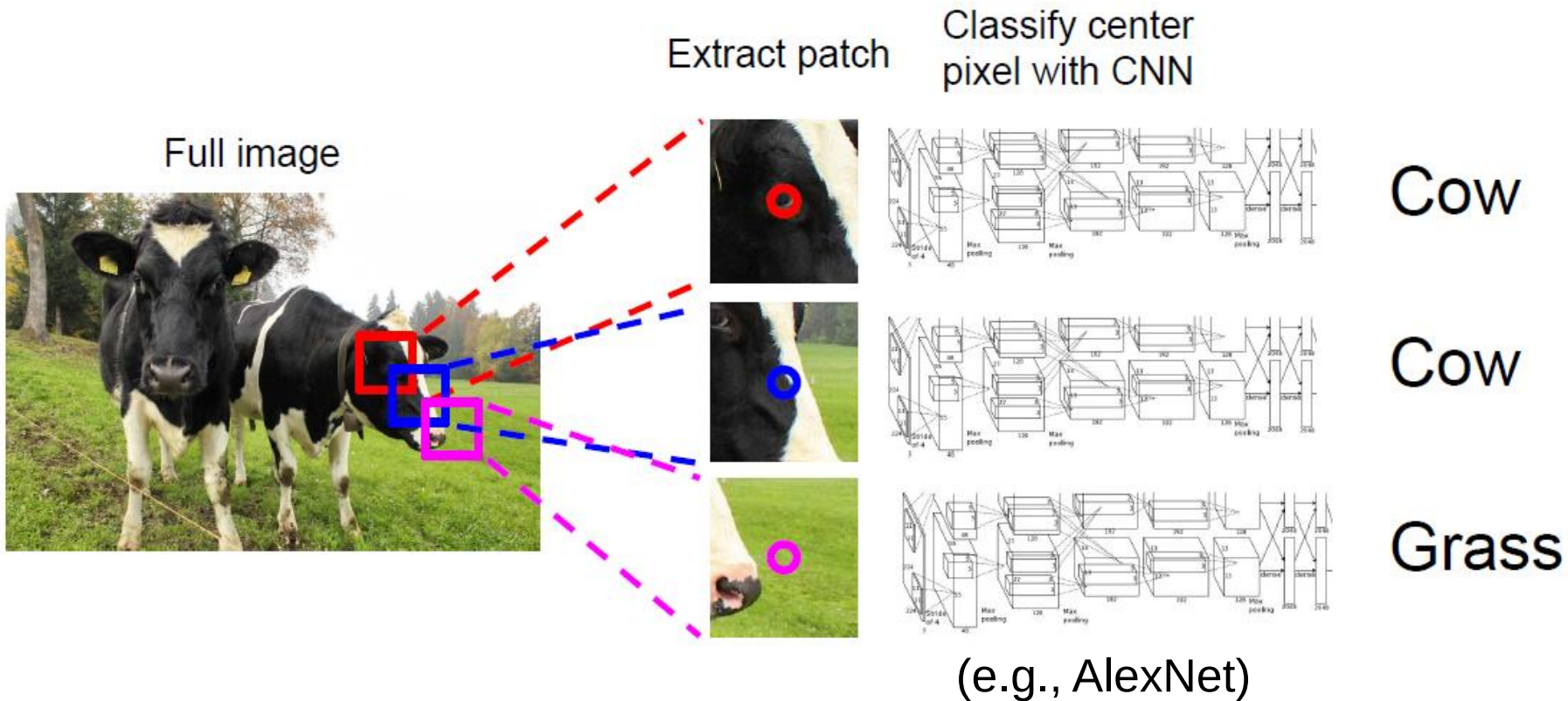
- CNNs for Object Detection
 - The R-CNN Family
 - The YOLO Family
- CNNs for Segmentation
 - Fully Convolutional Networks (FCN)
 - Encoder-Decoder architecture
 - Transpose convolutions
 - Skip connections
- CNNs for Human Pose Estimation
 - 2D Body pose estimation
 - 3D Body pose estimation
- CNNs for Matching
 - Siamese networks
 - Triplet loss

Semantic Segmentation

- Semantic Segmentation
 - Label each pixel in the image with a category label
 - Don't differentiate instances, only care about pixels
- Instance segmentation
 - Also give an instance label per pixel

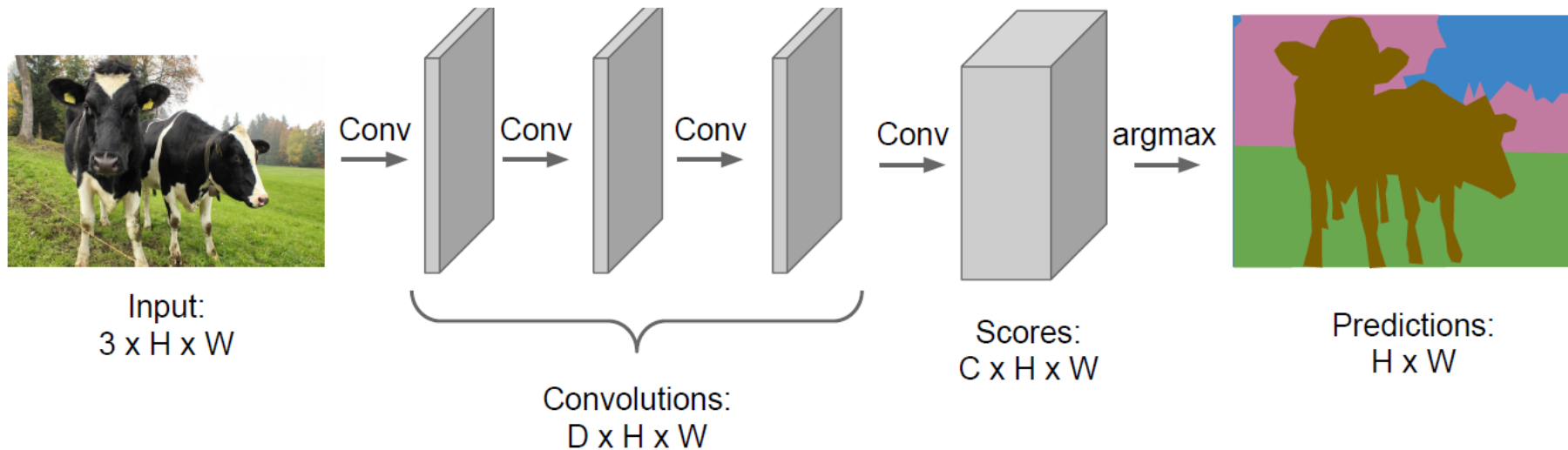


Segmentation Idea: Sliding Window



- Problem
 - Very inefficient
 - No reuse of features between shared patches

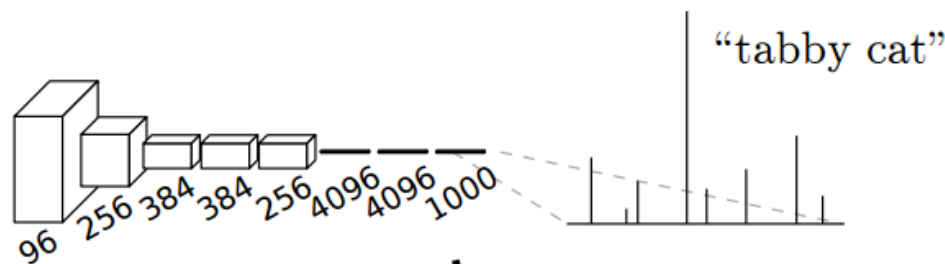
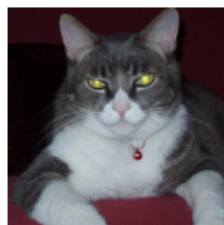
Segmentation Idea: Fully-Convolutional Nets



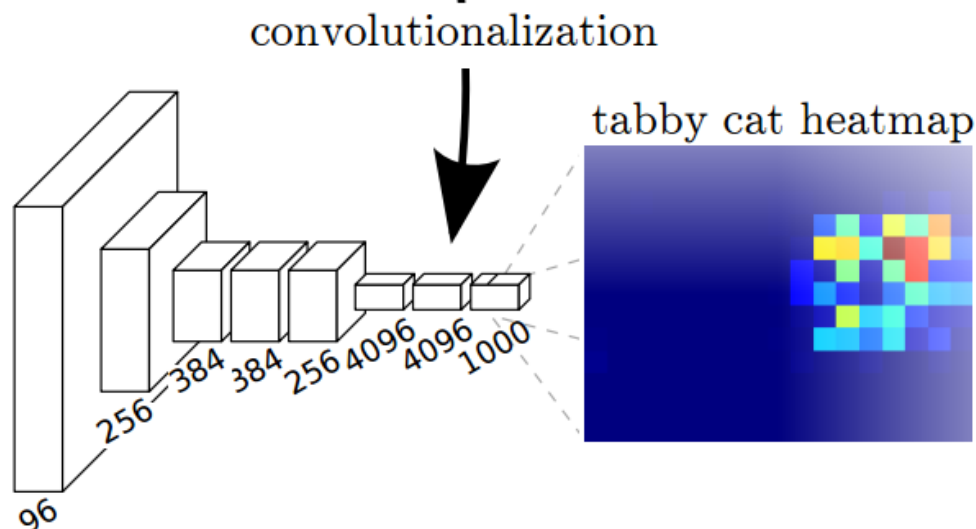
- Design a network as a sequence of convolutional layers
 - To make predictions for all pixels at once
 - **Fully Convolutional Networks (FCNs)**
 - All operations formulated as convolutions
 - Fully-connected layers become 1×1 convolutions
 - Advantage: can process arbitrarily sized images

CNNs vs. FCNs

- CNN



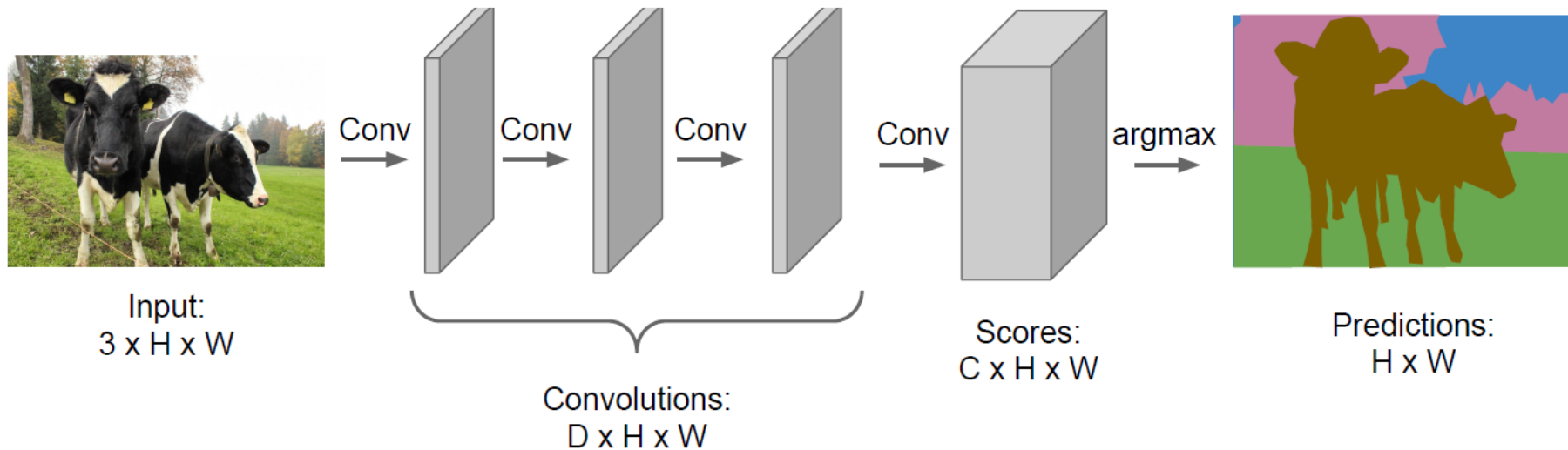
- FCN



- Intuition

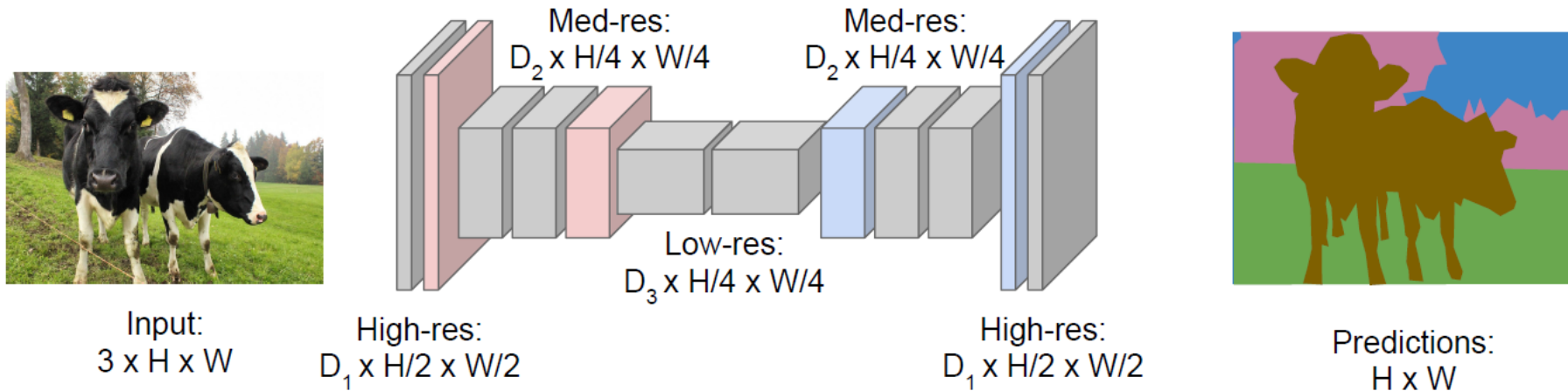
- Think of FCNs as performing a sliding-window classification, producing a heatmap of output scores for each class
- But: more efficient, since computations are reused between windows

Segmentation Idea: Fully-Convolutional Nets



- Design a network as a sequence of convolutional layers
 - To make predictions for all pixels at once
- Problem
 - Convolutions at original image resolution will be very expensive!

Segmentation Idea: Encoder-Decoder Nets



- Design a network as a sequence of convolutional layers
 - With **downsampling** and **upsampling** inside the network!
 - **Downsampling**
 - Pooling, strided convolution
 - **Upsampling**
 - ???

In-Network Upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4



1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

Output: 4 x 4

- Nearest-Neighbor
 - Simplest version
 - Problem: blocky output structure
- “Bed of Nails”
 - Preserve fine-grained structure of the output
 - Problem: fixed location for upsampled stimuli

In-Network Upsampling: “Max Unpooling”

Max Pooling

Remember which element was max!

1	2	6	3
3	5	2	1
1	2	2	1
7	3	4	8

Input: 4 x 4



5	6
7	8

Output: 2 x 2



Rest of the network

Max Unpooling

Use positions from pooling layer

1	2
3	4

Input: 2 x 2

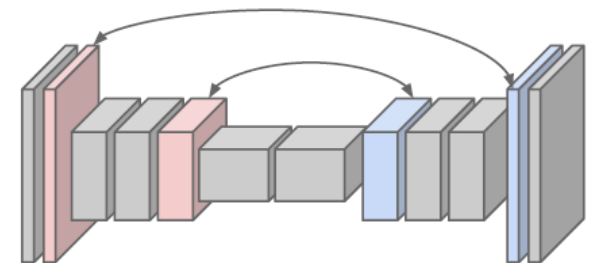


0	0	2	0
0	1	0	0
0	0	0	0
3	0	0	4

Output: 4 x 4

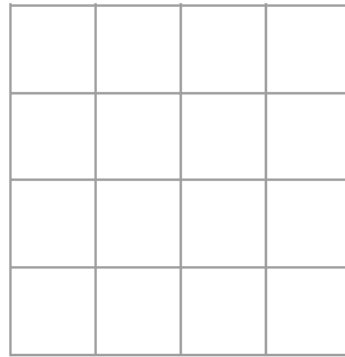
- Max Unpooling

- Use corresponding pairs of **downsampling** and **upsampling** layers together
- Remember which elements were max

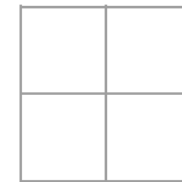


Learnable Upsampling: Transpose Convolution

- Recall: Normal convolution, stride 2, pad 1



Input: 4 x 4

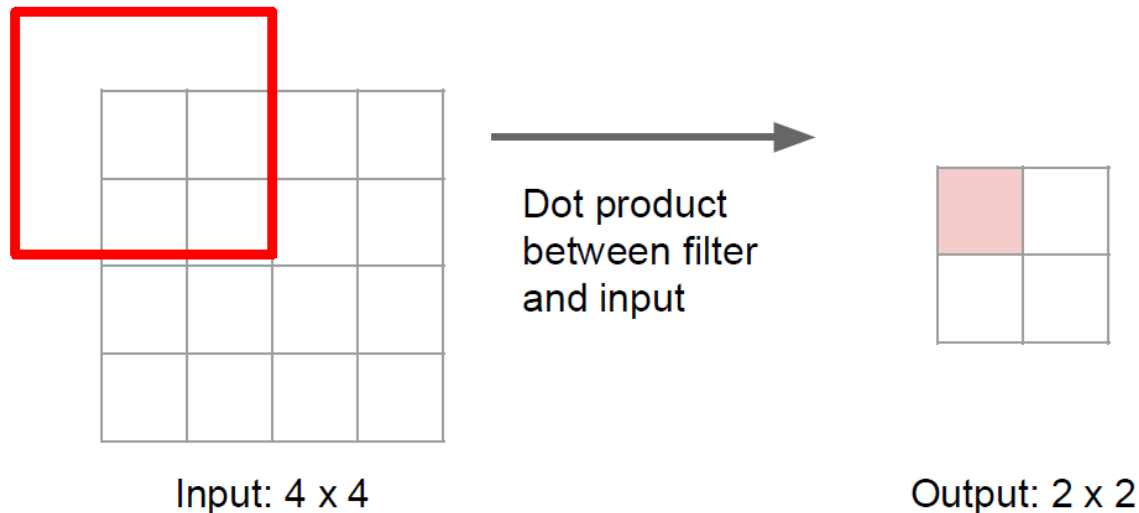


Output: 2 x 2

- Effect
 - Filter moves 2 pixels in the input for every one pixel in the output
 - Stride gives ration between movement in input and output

Learnable Upsampling: Transpose Convolution

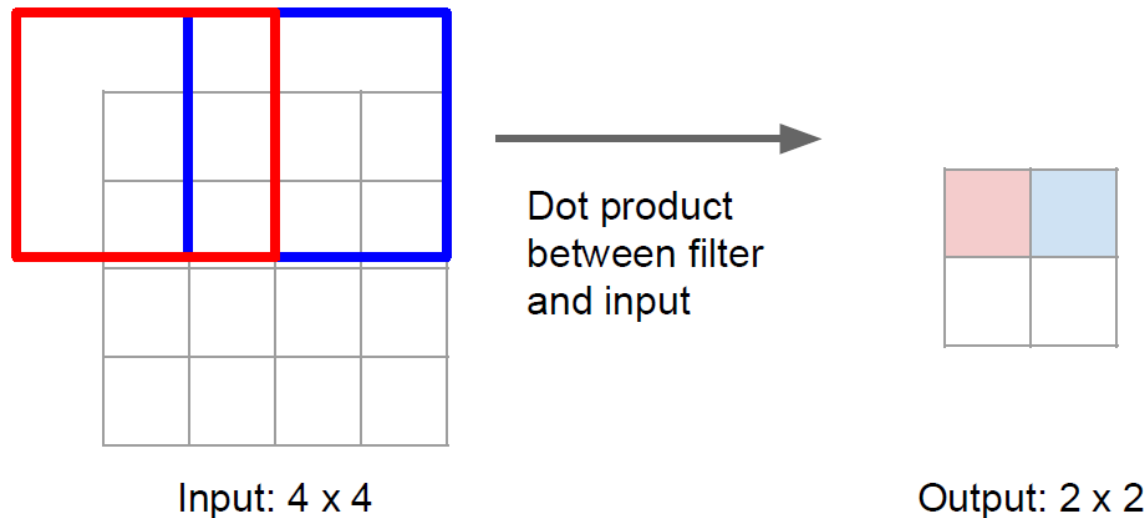
- Recall: Normal convolution, stride 2 pad 1



- Effect
 - Filter moves 2 pixels in the input for every one pixel in the output
 - Stride gives ration between movement in input and output

Learnable Upsampling: Transpose Convolution

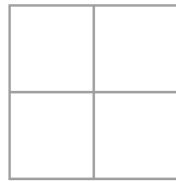
- Recall: Normal convolution, stride 2 pad 1



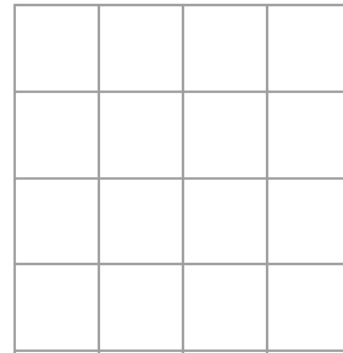
- Effect
 - Filter moves 2 pixels in the input for every one pixel in the output
 - Stride gives ration between movement in input and output

Learnable Upsampling: Transpose Convolution

- Now: 3x3 **transpose** convolution, stride 2 pad 1



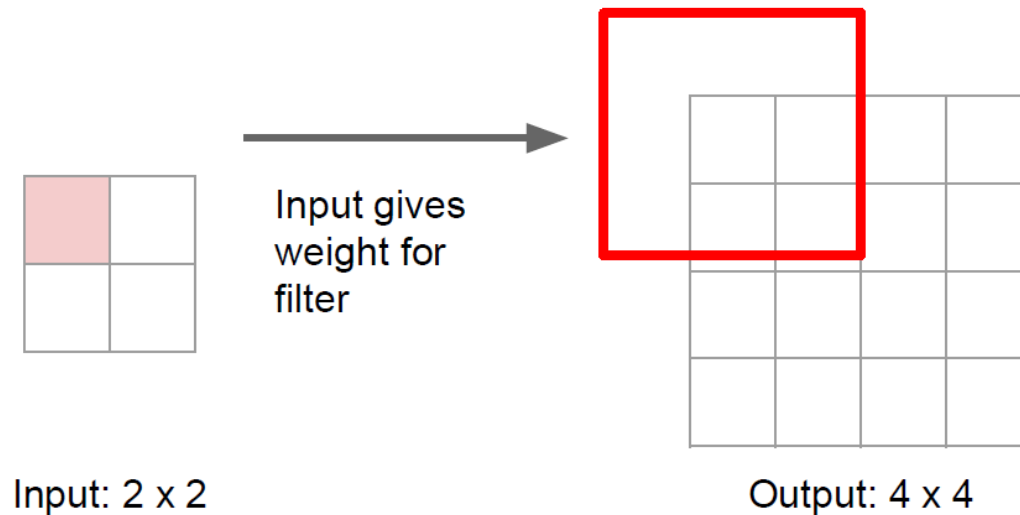
Input: 2 x 2



Output: 4 x 4

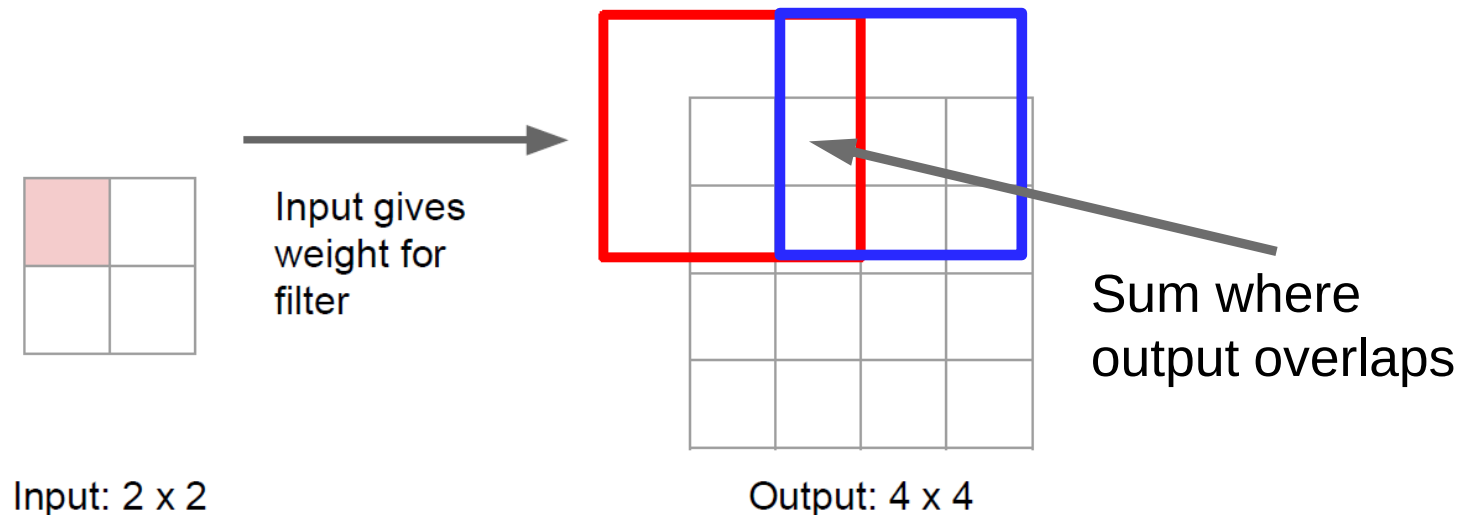
Learnable Upsampling: Transpose Convolution

- Now: 3x3 **transpose** convolution, stride 2 pad 1



Learnable Upsampling: Transpose Convolution

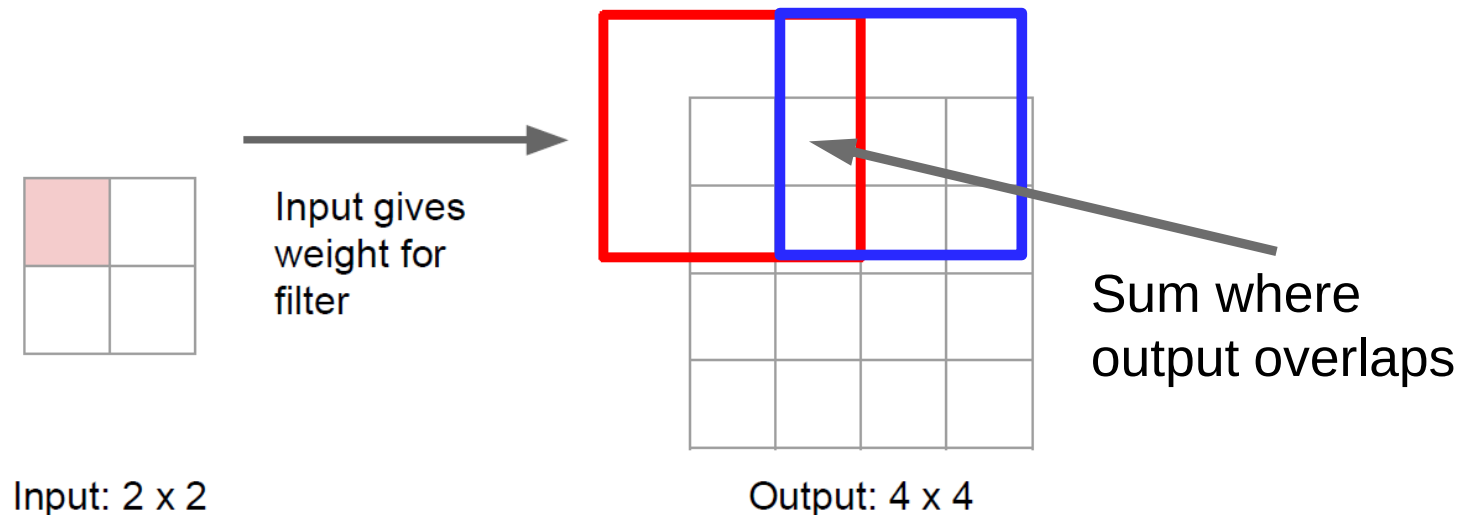
- Now: 3x3 **transpose** convolution, stride 2 pad 1



- Effect
 - Filter moves 2 pixels in the *output* for every one pixel in the *input*
 - Stride gives ration between movement in output and input

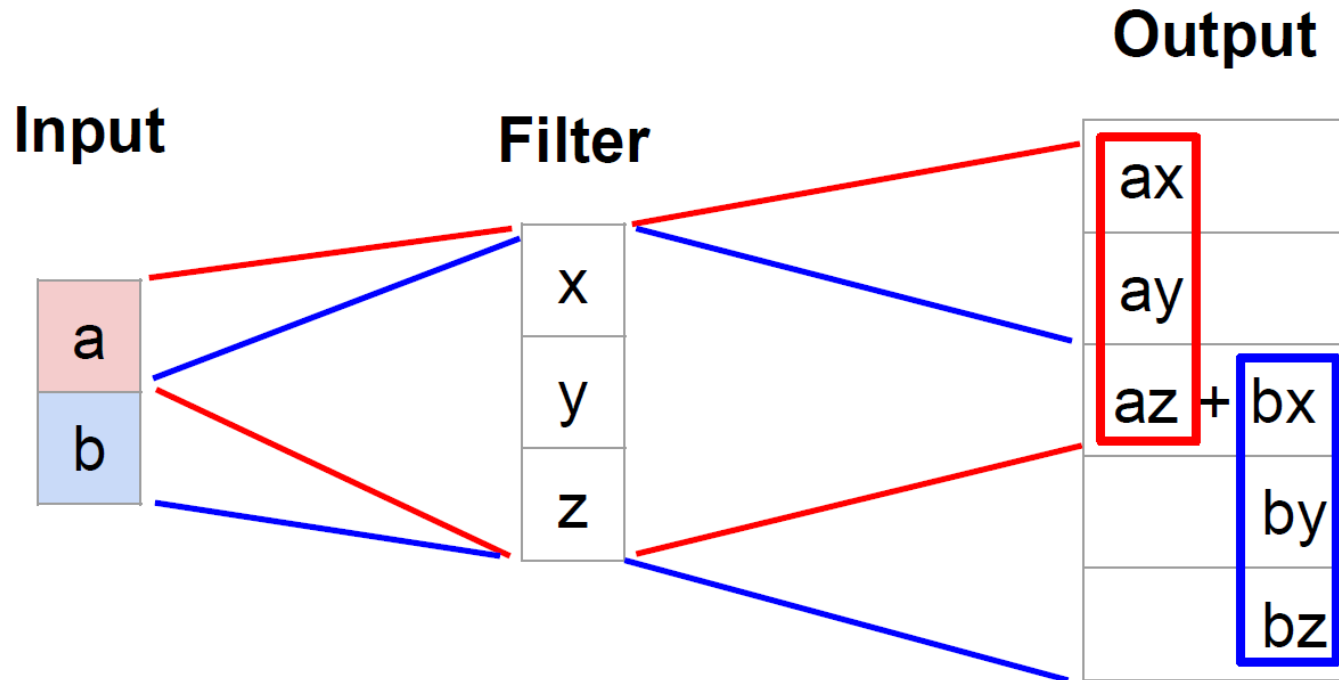
Learnable Upsampling: Transpose Convolution

- Now: 3x3 **transpose** convolution, stride 2 pad 1



- Other names
 - Deconvolution (bad)
 - Upconvolution
 - Fractionally strided convolution
 - Backward strided convolution

Learnable Upsampling: 1D Example



- Observations
 - Output contains copies of the filter weighted by the input, summing overlaps in the output
 - Need to crop one pixel from output to make output exactly 2x input

Convolution as Matrix Multiplication (1D Example)

- Express convolution in terms of matrix multiplication

$$\vec{x} * \vec{a} = X \vec{a}$$

- Example:

- 1D conv
- Kernel size = 3
- Stride 1, padding = 1

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & x & y & z & 0 & 0 \\ 0 & 0 & x & y & z & 0 \\ 0 & 0 & 0 & x & y & z \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ ax + by + cz \\ bx + cy + dz \\ cx + dy \end{bmatrix}$$

- Convolution transpose multiplies by the transpose of the same matrix

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

- When stride = 1, convolution transpose is just a regular convolution (with different padding rules)

$$\begin{bmatrix} x & 0 & 0 & 0 \\ y & x & 0 & 0 \\ z & y & x & 0 \\ 0 & z & y & x \\ 0 & 0 & z & y \\ 0 & 0 & 0 & z \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} ax \\ ay + bx \\ az + by + cx \\ bz + cy + dx \\ cz + dy \\ dz \end{bmatrix}$$

Convolution as Matrix Multiplication (1D Example)

- Express convolution in terms of matrix multiplication

$$\vec{x} * \vec{a} = X \vec{a}$$

- Example:

- 1D conv
- Kernel size = 3
- Stride 2, padding = 1

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

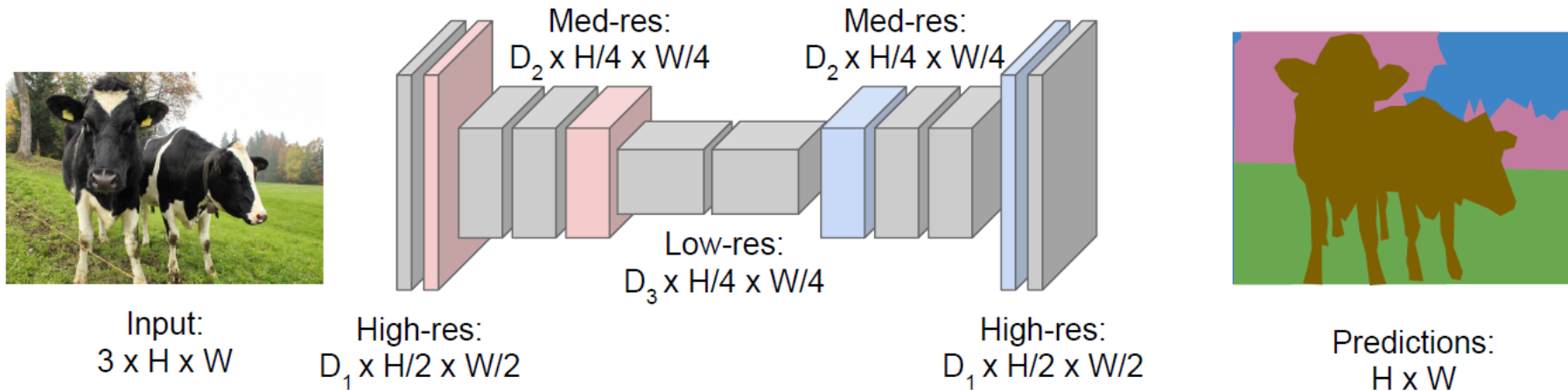
- Convolution transpose** multiplies by the transpose of the same matrix

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

- When stride > 1, convolution transpose is **no longer a normal convolution!**

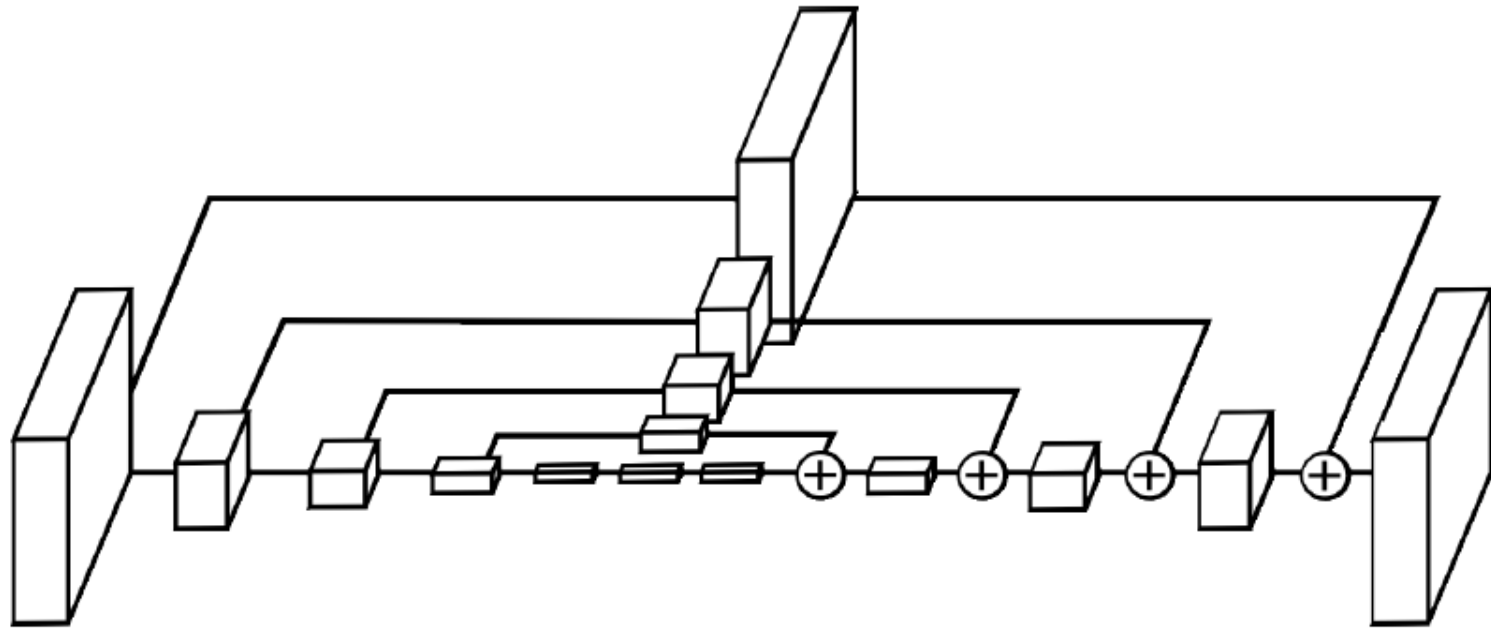
$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

Segmentation Idea: Encoder-Decoder Nets



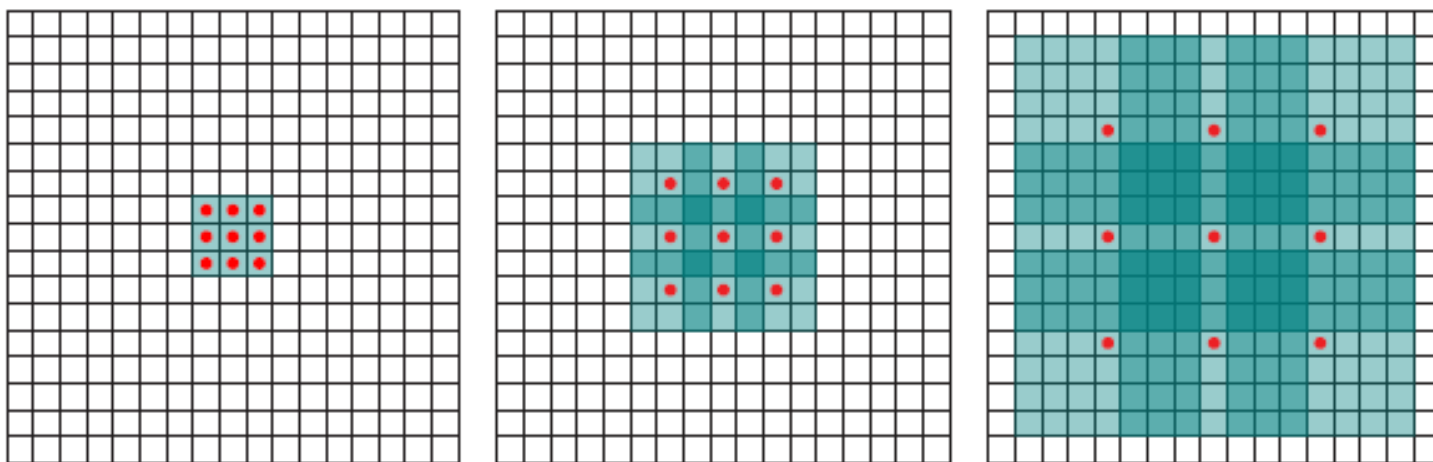
- Design a network as a sequence of convolutional layers
 - With **downsampling** and **upsampling** inside the network!
 - **Downsampling**
 - Pooling, strided convolution
 - **Upsampling**
 - Unpooling or strided transpose convolution

Extension: Skip Connections



- Encoder-Decoder Architecture with skip connections
 - Problem: downsampling loses high-resolution information
 - Use skip connections to preserve this higher-resolution information

Extension: Dilated Convolutions



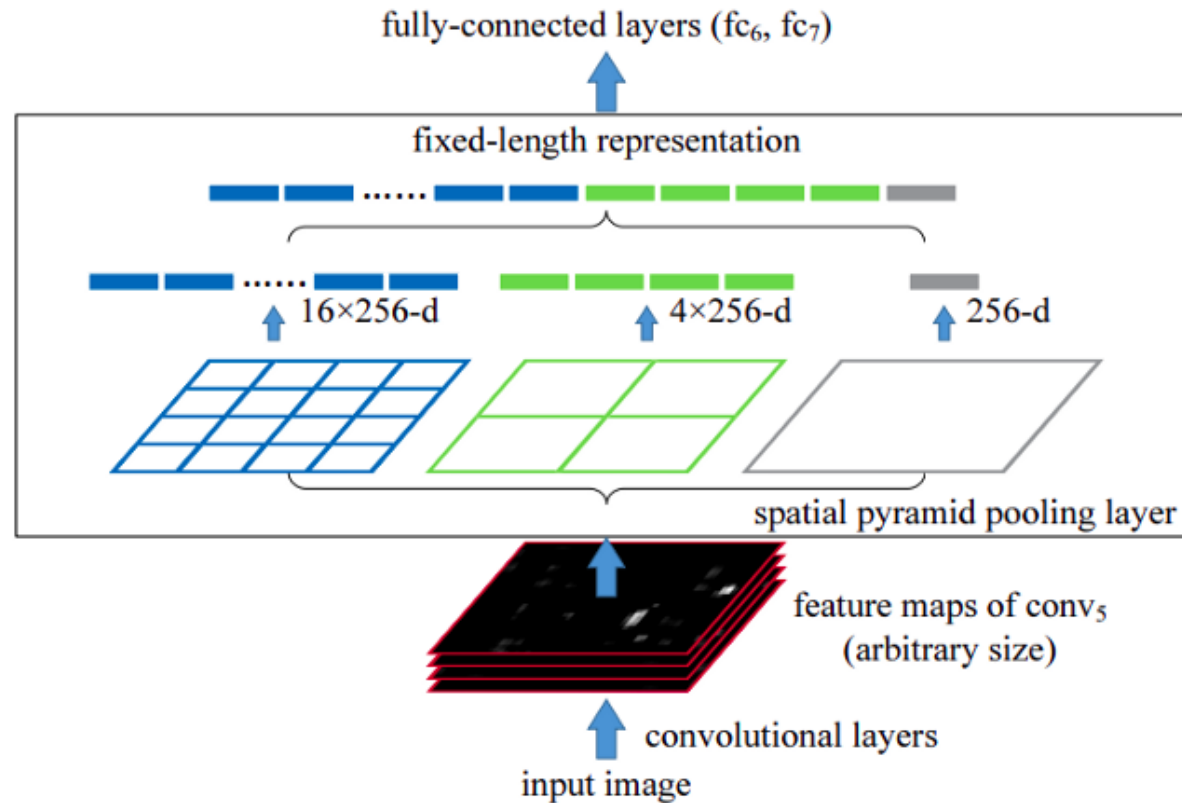
- Also known as: **à trous convolutions** or **atrous convolutions**
 - Simple way to increase the receptive field size without introducing additional parameters (filter size stays the same)

$$(F \star_l k)(\mathbf{p}) = \sum_{\mathbf{s} + l\mathbf{t} = \mathbf{p}} F(\mathbf{s}) k(\mathbf{t}) \quad \text{with dilation factor } l$$

- Technique plays a key role in the *algorithme à trous* for wavelet decomposition [Holschneider et al., 1987]

Fisher Yu, Vladlen Koltun, [Multi-Scale Context Aggregation by Dilated Convolutions](#), ICLR 2016

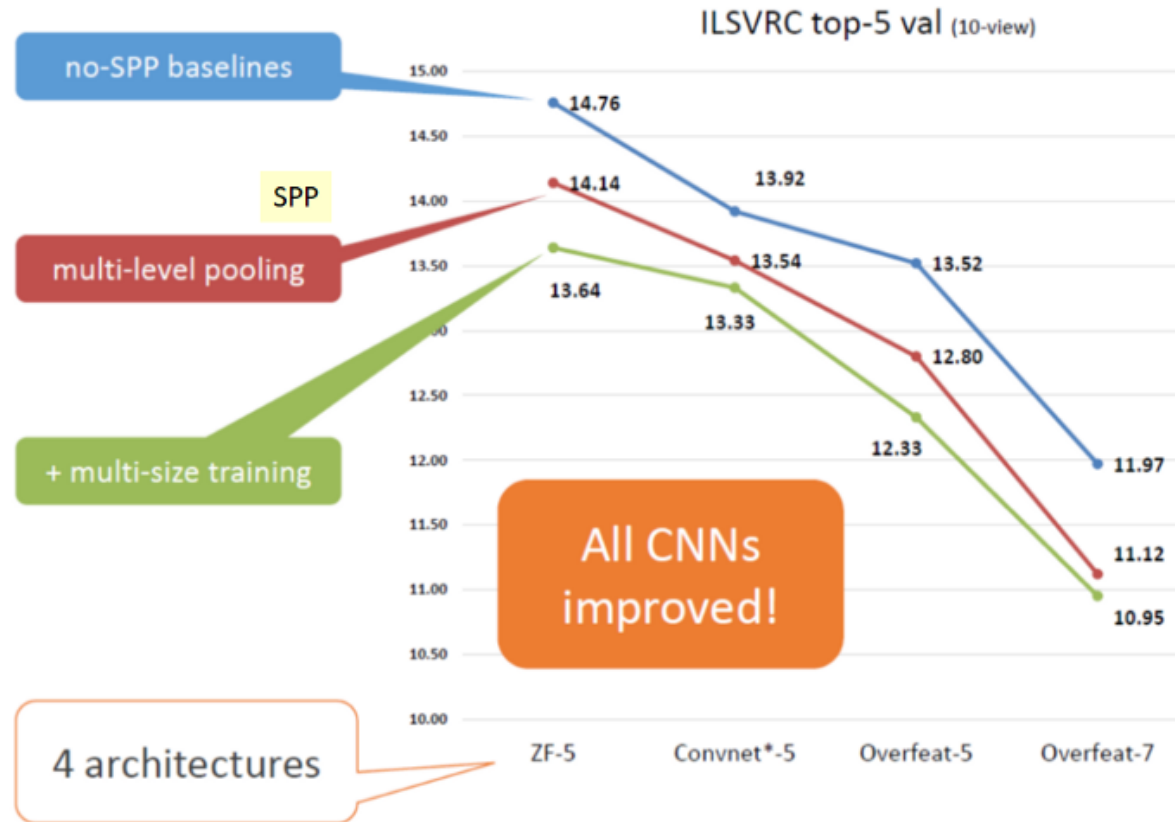
Extension: Spatial Pyramid Pooling (SPP)



- Multi-scale feature representation
 - Concatenated to a fixed-length vector

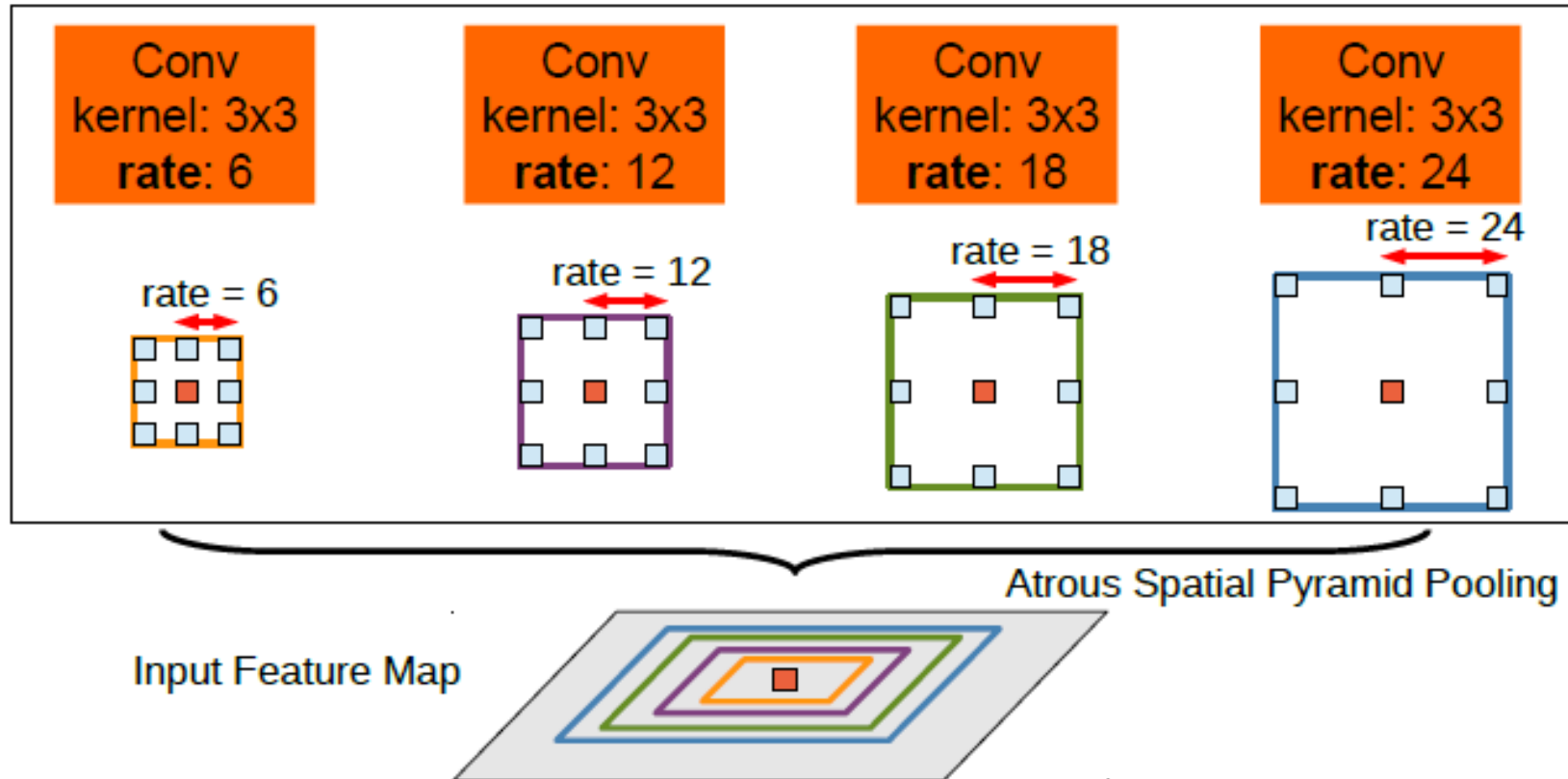
K. He, X. Zhang, S. Ren, J. Sun, [Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition](#), ECCV 2014.

Spatial Pyramid Pooling (SPP)



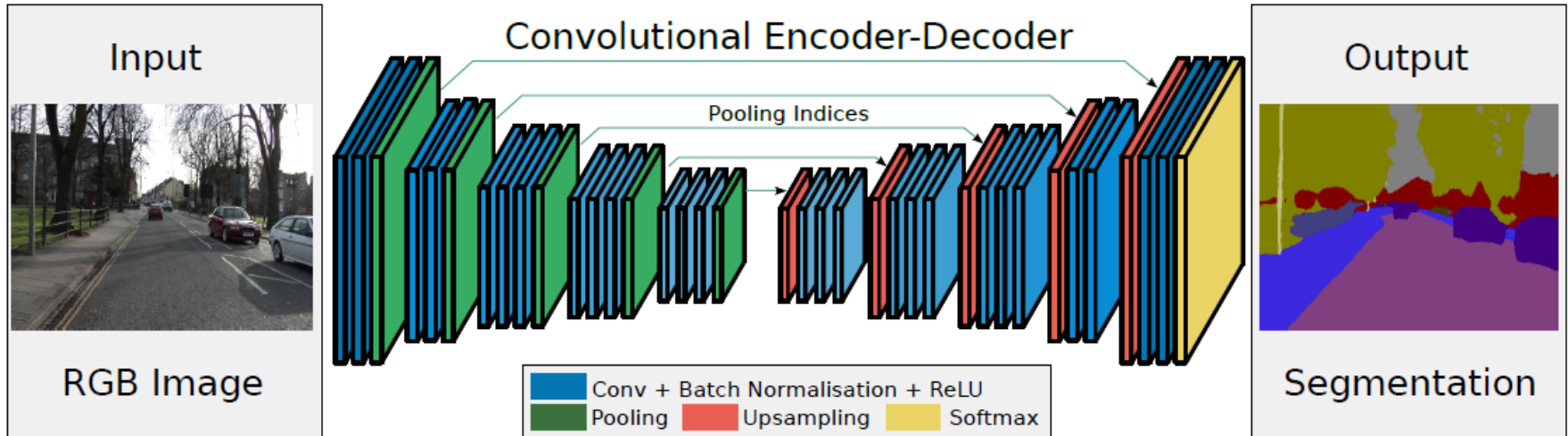
- Effect
 - Improved recognition performance for variety of architectures
 - SPP modules are used in many current architectures

Atrous Spatial Pyramid Pooling (ASPP)



- Extension of the SPP concept
 - Using **dilated convolutions** instead of regular convolutions
 - Effect: increased receptive field without extra computation

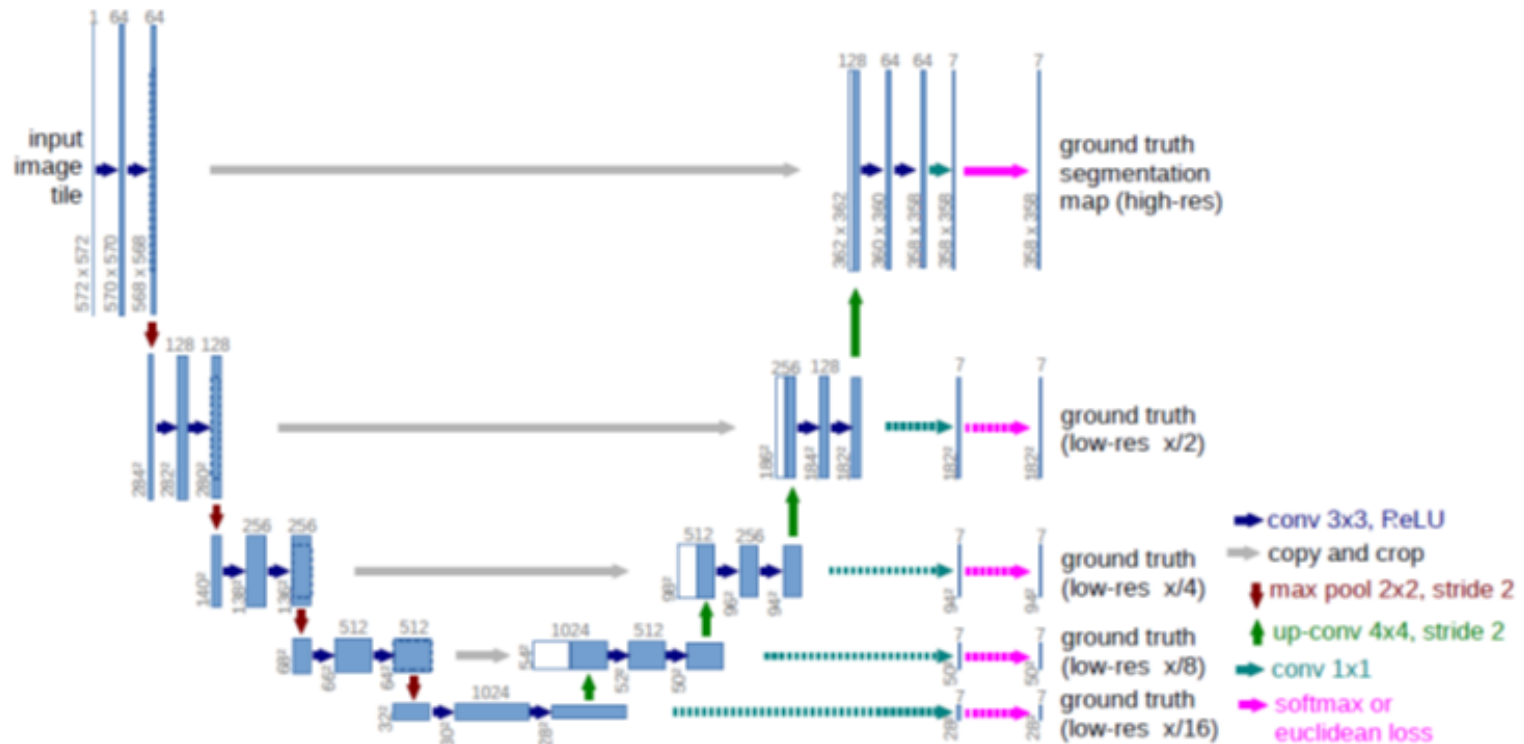
Example: SegNet



- SegNet
 - Encoder-Decoder architecture with skip connections
 - Encoder based on VGG-16
 - Decoder using Max Unpooling
 - Output with K-class Softmax classification

V. Badrinarayanan, A. Kendall, R. Cipolla, [SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation](#), arXiv 1511.00561, IEEE Trans. PAMI 2017.

Example: U-Net

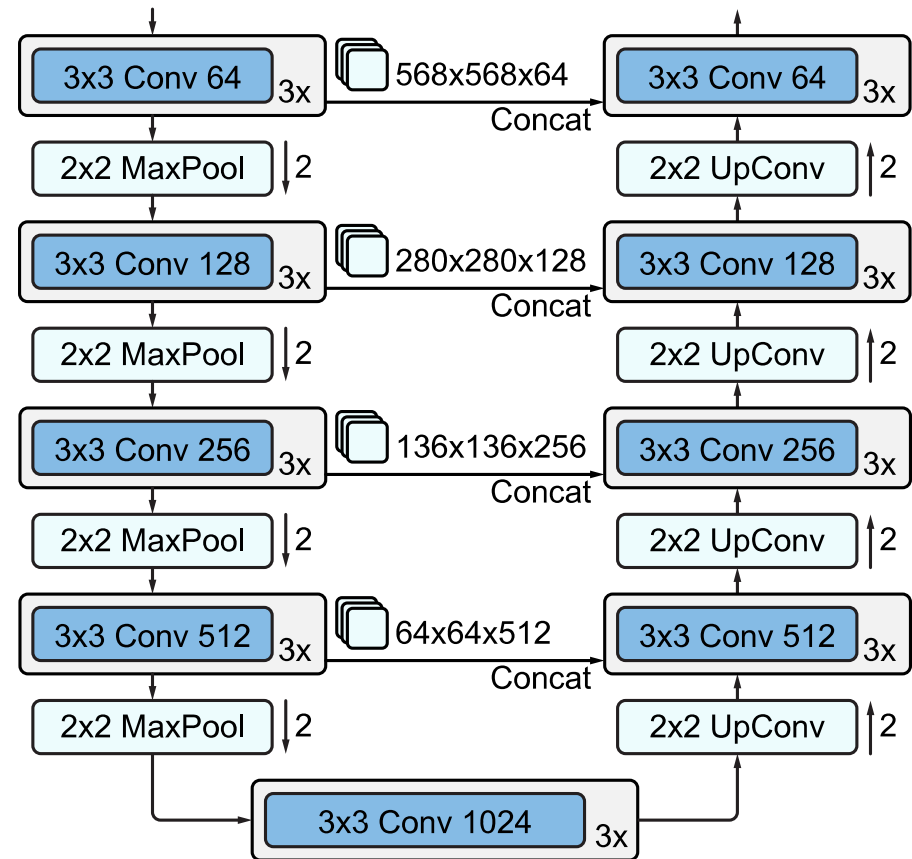


- U-Net
 - Similar idea, popular in biomedical image processing
 - Encoder-Decoder architecture with skip connections

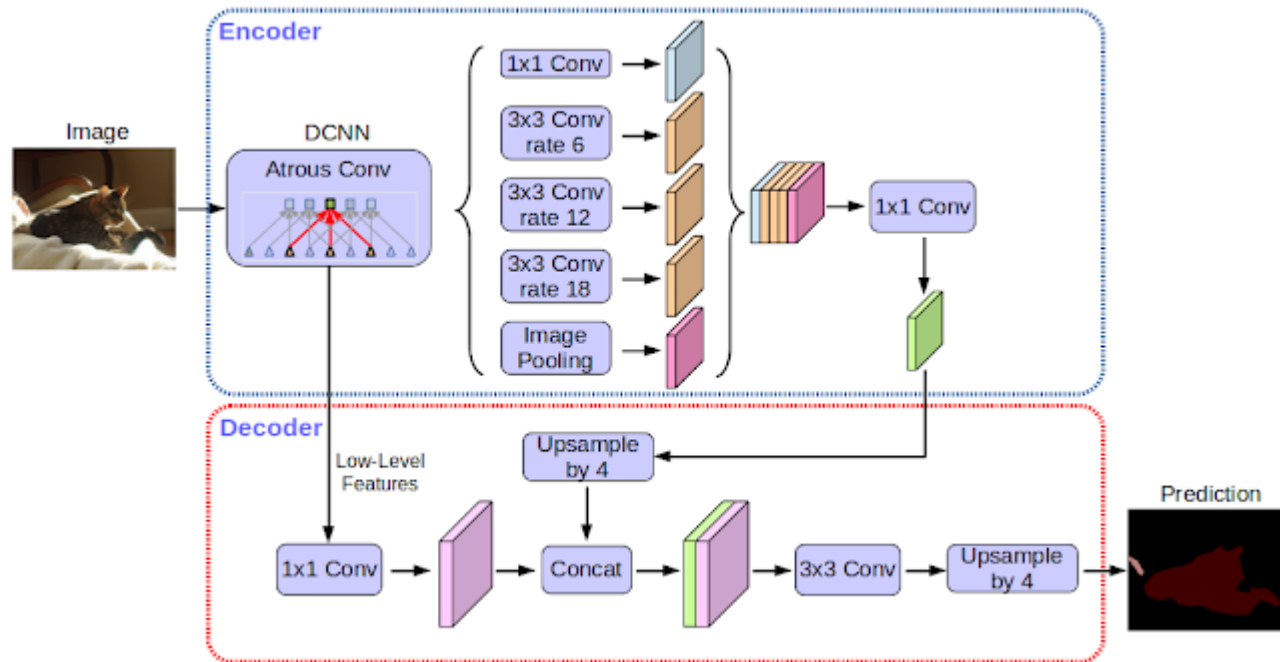
O. Ronneberger, P. Fischer, T. Brox, [U-Net: Convolutional Networks for Biomedical Image Segmentation](#), MICCAI 2015

Example: U-Net

- Encoder-Decoder structure
 - Encoder uses max pooling
 - Decoder uses transpose convolutions
 - Skip connections
 - Preserve high-resolution information



Example: DeepLabv3+



- Architecture designed for semantic segmentation
 - Uses **atrous spatial pyramid pooling (ASPP)** in the encoder
 - Simple decoder module
 - Depth-wise separable convolutions for efficiency

L-C. Chen, G. Papandreou, F. Schroff, H. Adam, [DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolutions, and Fully Connected CRFs](#), IEEE Trans. PAMI, 2017

Semantic Segmentation



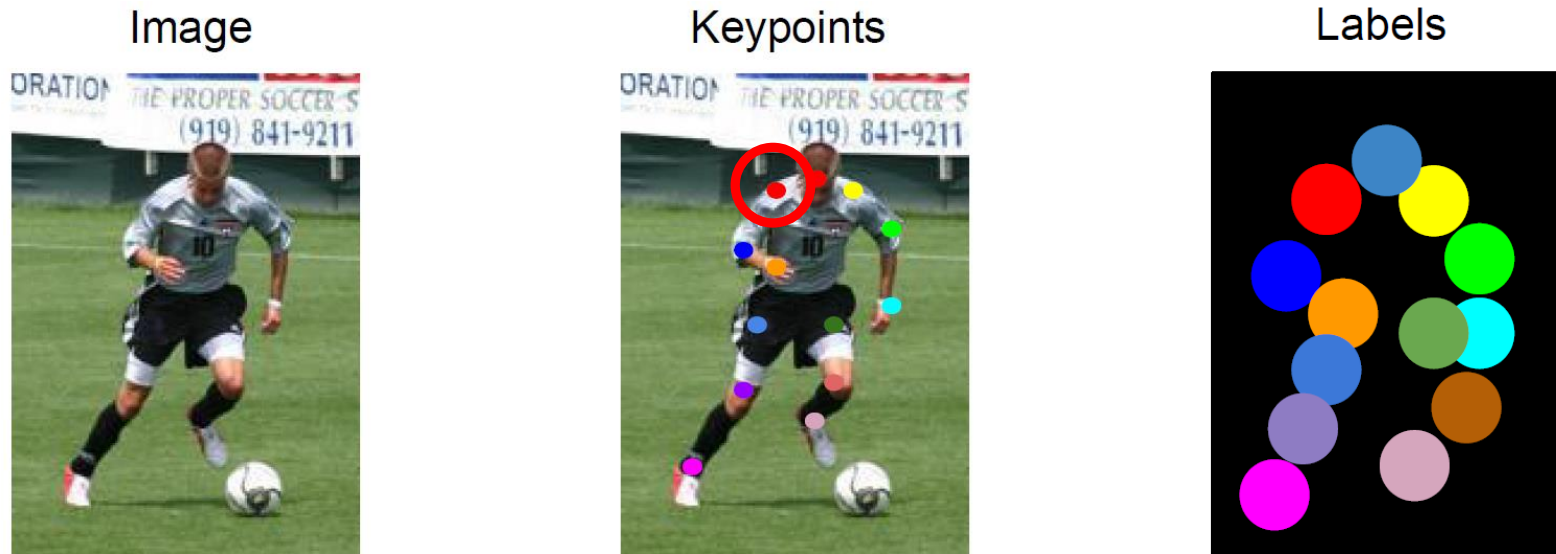
- Recent results
 - Based on an extension of ResNets for high-resolution segmentation

Topics of This Lecture

- CNNs for Object Detection
 - The R-CNN Family
 - The YOLO Family
- CNNs for Segmentation
 - Fully Convolutional Networks (FCN)
 - Encoder-Decoder architecture
 - Transpose convolutions
 - Skip connections
- CNNs for Human Pose Estimation
 - 2D Body pose estimation
 - 3D Body pose estimation
- CNNs for Matching
 - Siamese networks
 - Triplet loss

CNNs for Human Pose Estimation

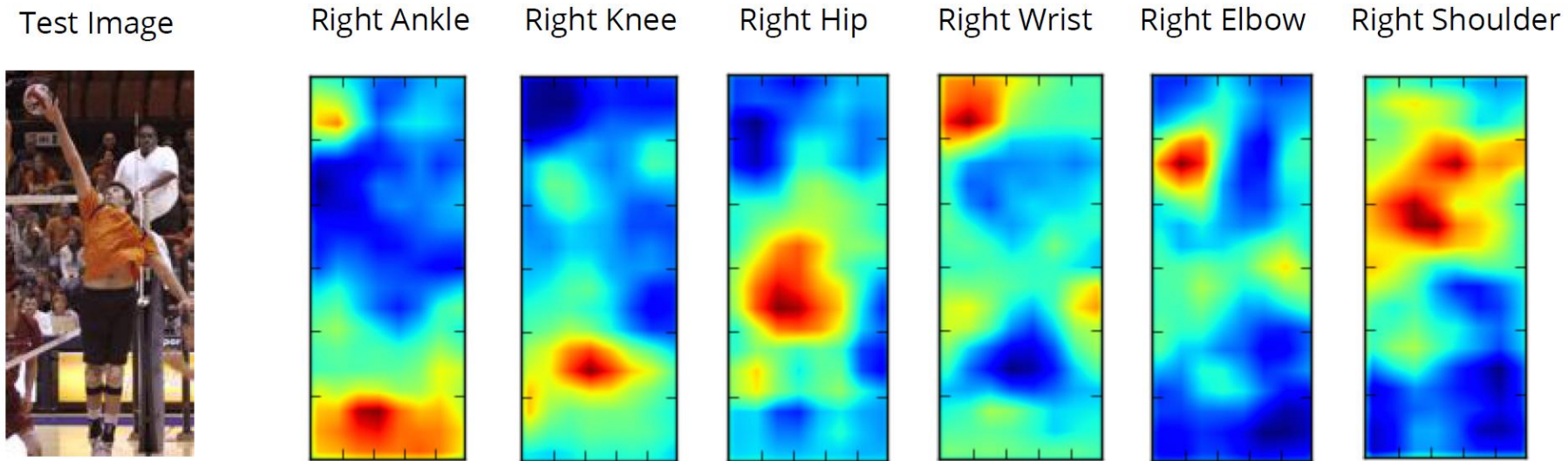
- Input data



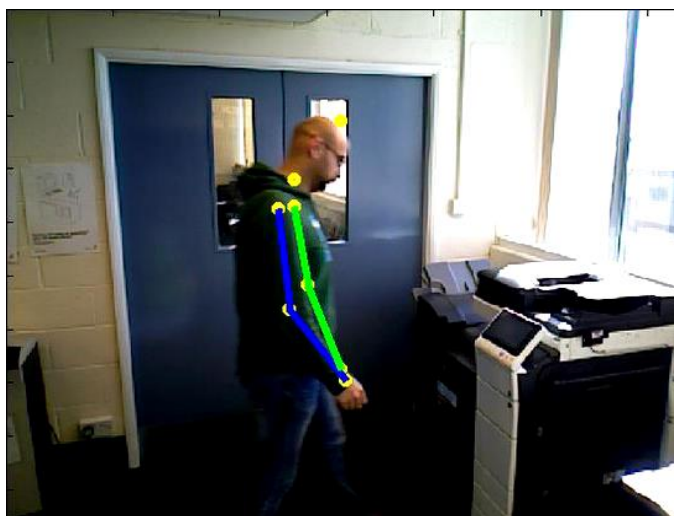
- Task setup

- Annotate images with keypoints for skeleton joints
- Define a target disk around each keypoint with radius r
- Set the ground-truth label to 1 within each such disk
- Infer heatmaps for the joints as in semantic segmentation

Heat Map Predictions from FCN



Example Results: Human Pose Estimation



Example Results: OpenPose

OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields

Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, Yaser Sheikh
Carnegie Mellon University

Z. Cao, G. Hidalgo Martinez, T. Simon, S.-E. Wei, [OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields](#), IEEE Trans. PAMI, 2019. ([code available here](#))

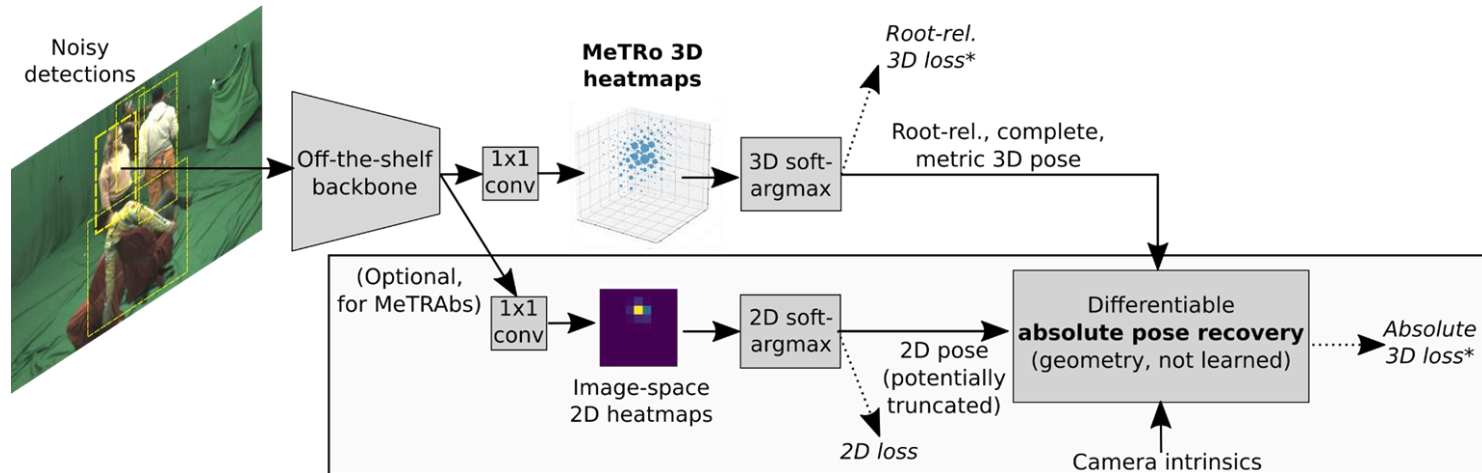
3D Human Body Pose Estimation



MeTRAbs

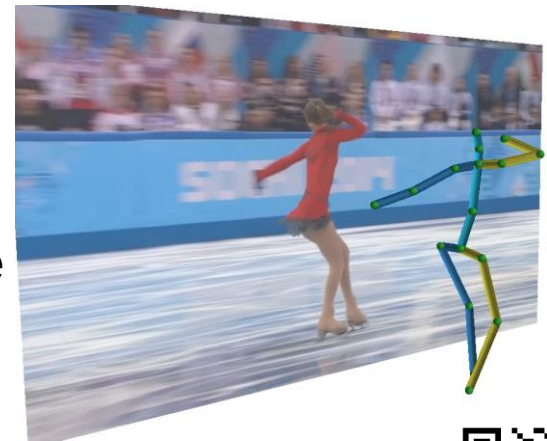


Architecture: CNN + Differentiable Geometry



- 3D Body Pose Estimation

- Metric-Scale Truncation-Robust Heatmaps for 3D human pose estimation (MeTRo)
- Robust to truncation, suitable for close range
- Very simple network design, extremely fast (can process >500 detections/s on a desktop GPU)



I. Sarandi, T. Linder, K.O. Arras, B. Leibe, [MeTRAbs: Metric-Scale Truncation-Robust Heatmaps for Absolute 3D Human Pose Estimation](#), IEEE Trans. BIOM, 2020.

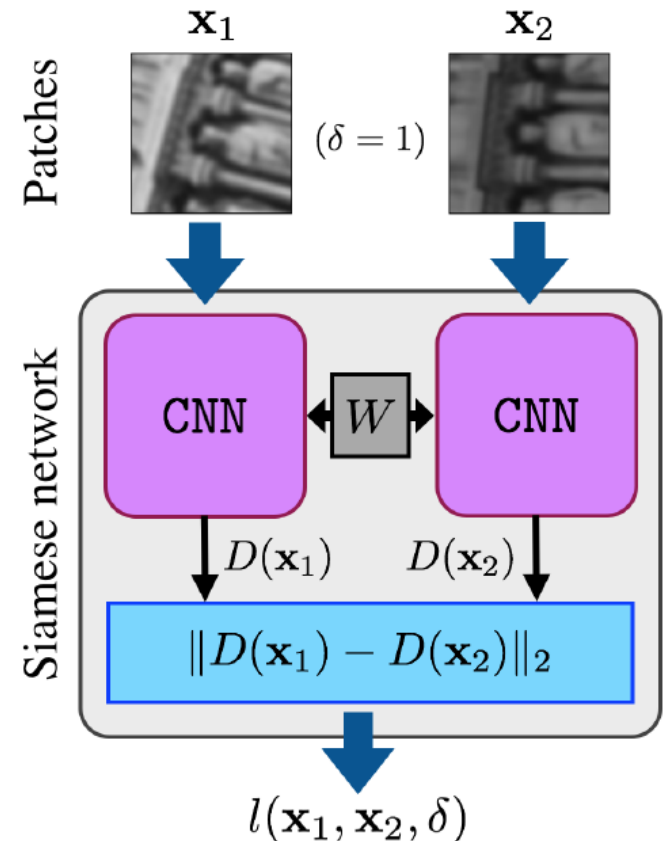


Topics of This Lecture

- CNNs for Object Detection
 - The R-CNN Family
 - The YOLO Family
- CNNs for Segmentation
 - Fully Convolutional Networks (FCN)
 - Encoder-Decoder architecture
 - Transpose convolutions
 - Skip connections
- CNNs for Human Pose Estimation
 - 2D Body pose estimation
 - 3D Body pose estimation
- CNNs for Matching
 - Siamese networks
 - Triplet loss

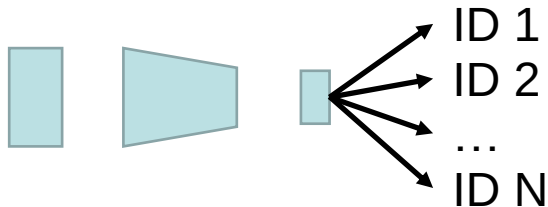
Learning Similarity Functions

- Siamese Network
 - Present the two stimuli to two identical copies of a network (with shared parameters)
 - Train them to output similar values if the inputs are (semantically) similar.
- Used for many matching tasks
 - Face identification
 - Stereo estimation
 - Optical flow
 - ...
- *How can we train a network to achieve this?*



Types of Models used for Matching Tasks

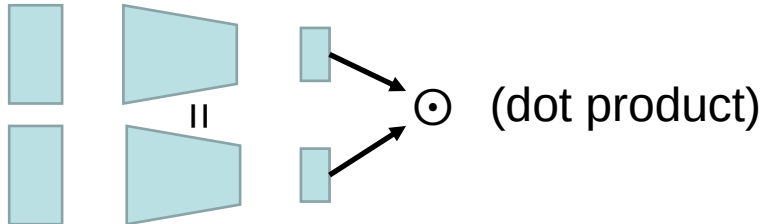
- Identification models (I)



Training

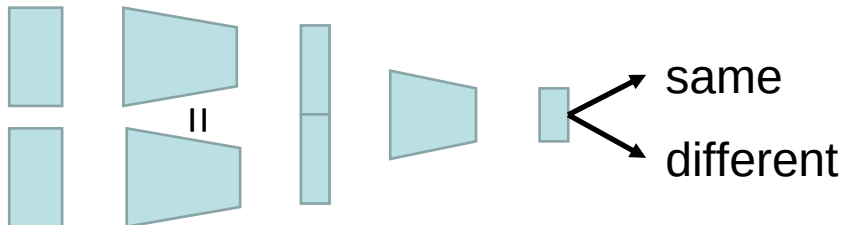
Multi-class
classification loss

- Embedding models (E)



Large-margin loss,
Triplet loss

- Verification models (V)



Two-class
classification loss

Triplet Loss Networks

- Learning a discriminative embedding
 - Present the network with triplets of examples

Negative



Anchor

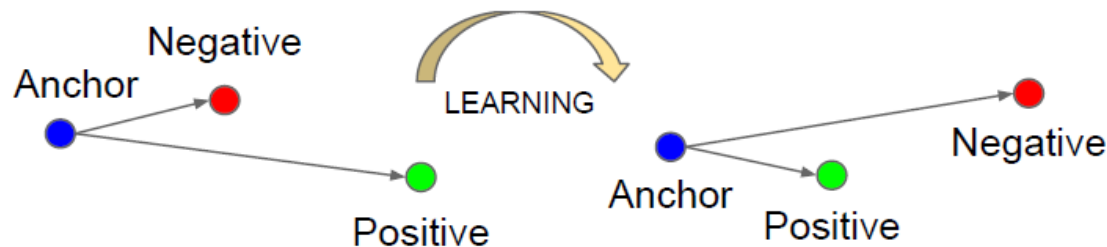


Positive



- Apply triplet loss to learn an embedding $f(\cdot)$ that groups the positive example closer to the anchor than the negative one.

$$\|f(x_i^a) - f(x_i^p)\|_2^2 < \|f(x_i^a) - f(x_i^n)\|_2^2$$



⇒ Used with great success in Google's FaceNet face identification

Triplet Loss – Practical Implementation

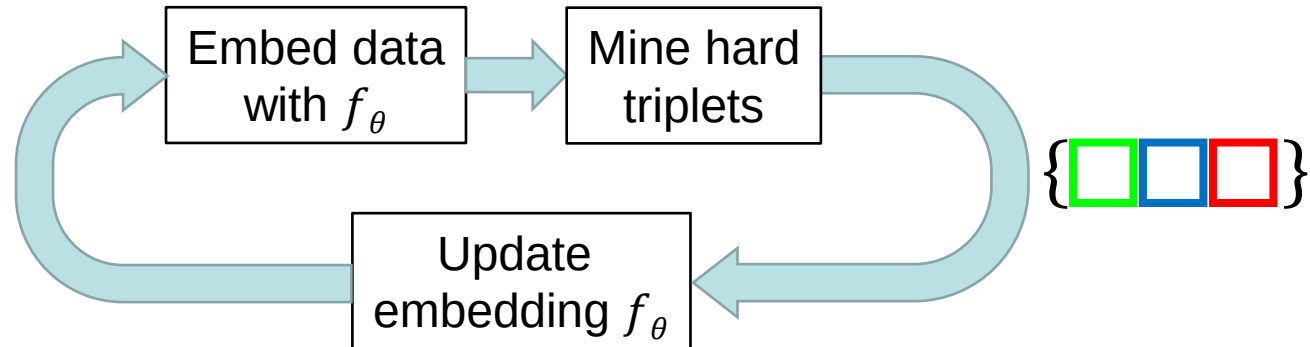
- Triplet loss formulation (using distances $D_{a,p} = d(x_a, x_p)$)

$$\mathcal{L}_{\text{tri}}(\theta) = \sum_{\substack{a,p,n \\ y_a=y_p \neq y_n}} [m + D_{a,p} - D_{a,n}]_+$$

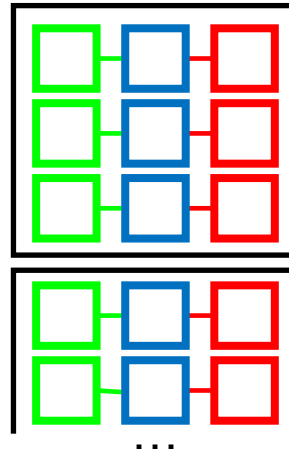
- Practical Issue: How to select the triplets?
 - The number of possible triplets grows cubically with the dataset size.
 - Most triplets are uninformative
 - ⇒ Mining hard triplets becomes crucial for learning.
 - ⇒ Actually want *medium-hard* triplets for best training efficiency
- Popular solution: Offline hard triplet mining
 - Process the dataset to find hard triplets
 - Use those for learning
 - Iterate

Offline Hard Triplet Mining

- Considerable effort needed



- Using the triplets for learning
 - Minibatch learning

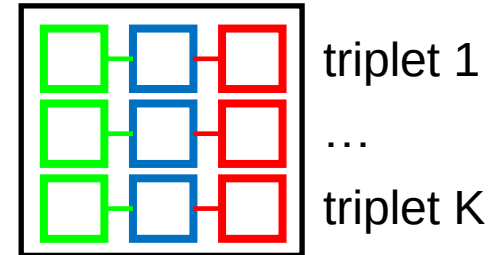


This is a very
wasteful design!

Better: Online Hard Triplet Mining

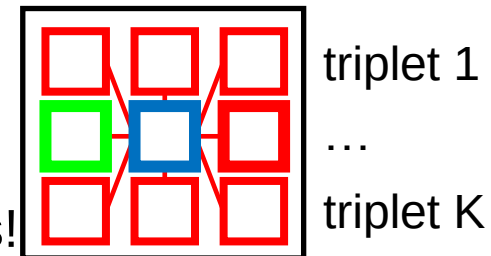
- Core idea

- The minibatch contains many more potential triplets than the ones that were mined!
- Why not make use of those also?



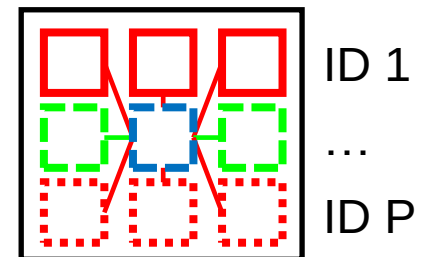
- Possible improvement

- Each member of another triplet becomes an additional negative candidate
- But: need both hard negatives *and* hard positives!

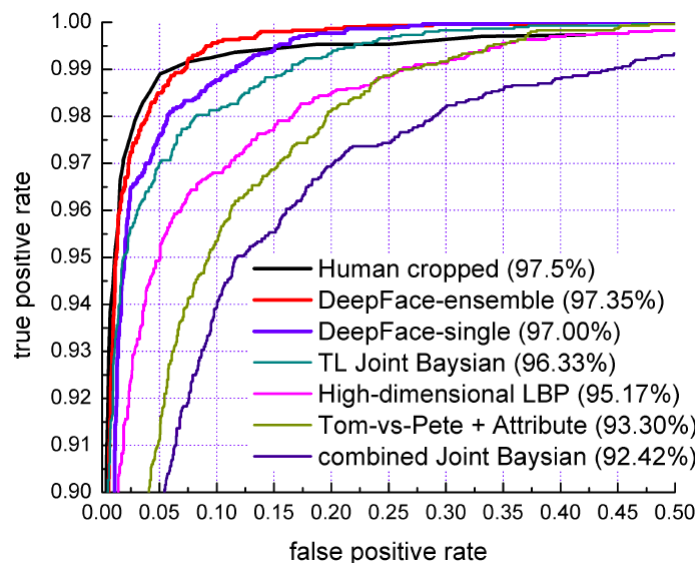
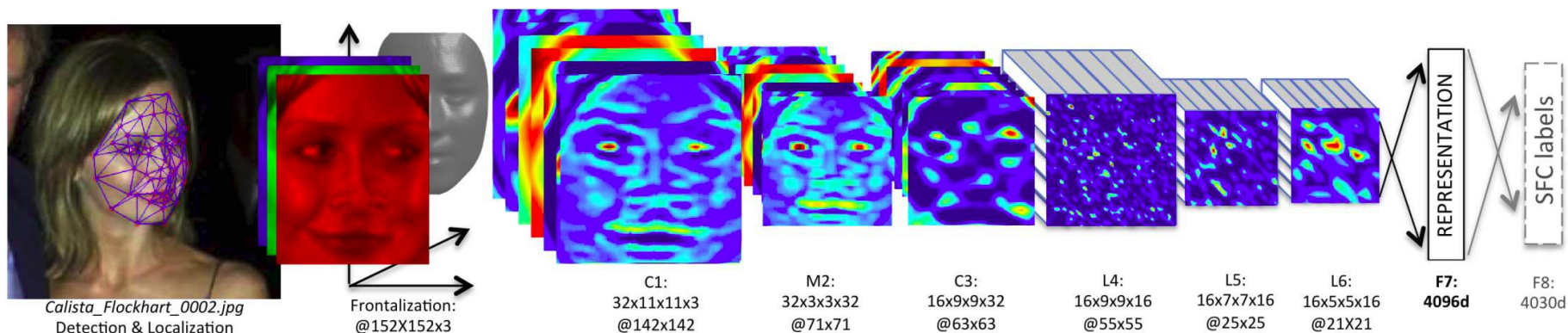


- Better design

- Sample K images from P classes (=people) for each minibatch
- Triplets are only constructed within the minibatch

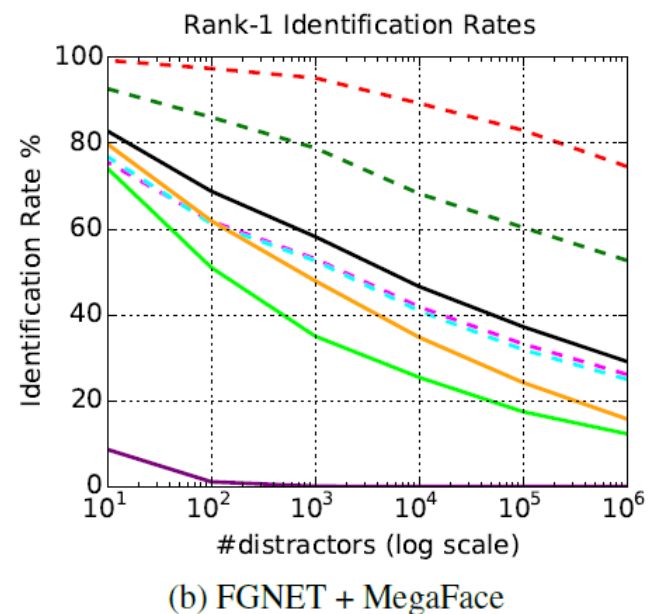
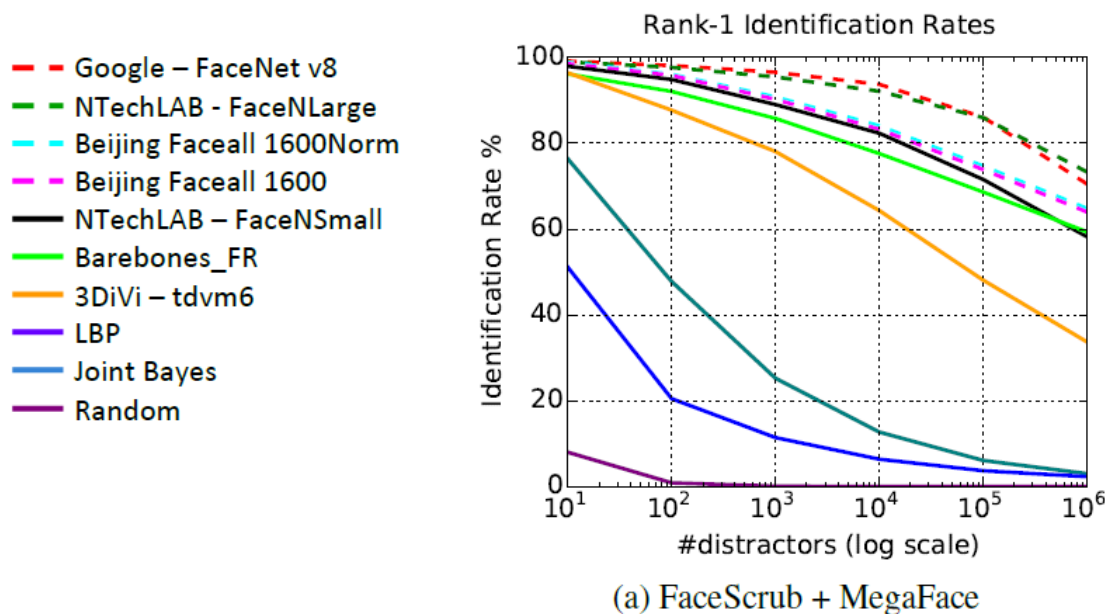


Applications: Face Identification



Y. Taigman, M. Yang, M. Ranzato, L. Wolf, [DeepFace: Closing the Gap to Human-Level Performance in Face Verification](#), CVPR 2014

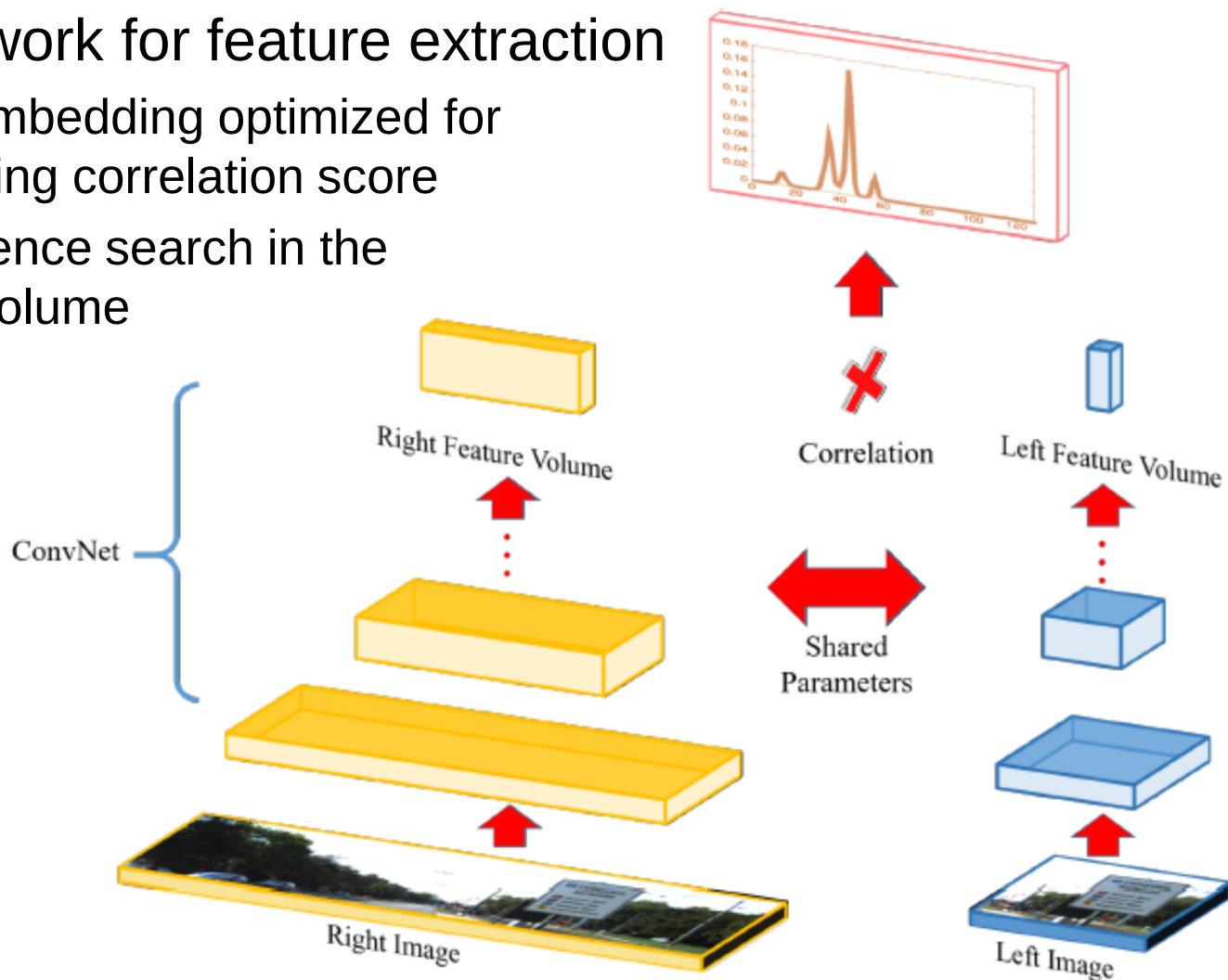
Face Identification Results



- Rank-1 identification rates of state-of-the-art algorithms
 - Performance with varying number of distractors in the gallery set
 - FaceNet: trained on 500M photos of 10M people

Applications: Stereo Matching

- Siamese network for feature extraction
 - Learns an embedding optimized for matching using correlation score
 - Correspondence search in the correlation volume



References: Computer Vision Tasks

- Object Detection

- R. Girshick, J. Donahue, T. Darrell, J. Malik, Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation, CVPR 2014.
- S. Ren, K. He, R. Girshick, J. Sun, [Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks](#), NIPS 2015.
- K. He, G. Gkioxari, P. Dollar, R. Girshick, [Mask R-CNN](#), arXiv 1703.06870.
- J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You Only Look Once: Unified Real-Time Object Detection, CVPR 2016.
- W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C-Y. Fu, A.C. Berg, SSD: Single Shot Multi Box Detector, ECCV 2016.

References: Computer Vision Tasks

- Semantic Segmentation

- J. Long, E. Shelhamer, T. Darrell, [Fully Convolutional Networks for Semantic Segmentation](#), CVPR 2015.
- O. Ronneberger, P. Fischer, T. Brox, [U-Net: Convolutional Networks for Biomedical Image Segmentation](#), MICCAI 2015
- V. Badrinarayanan, A. Kendall, R. Cipolla, [SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation](#), arXiv 1511.00561, IEEE Trans. PAMI 2017.
- T-Y. Lin P. Dollar, R. Girshick, K. He, B. Hariharan, S. Belongie, [Feature Pyramid Networks for Object Detection](#), CVPR 2017.
- L-C. Chen, G. Papandreou, F. Schroff, H. Adam, [Rethinking Atrous Convolutions for Semantic Segmentation](#), arXiv 1706.05587 2017.

References: Computer Vision Tasks

- Human Body Pose Estimation

- A. Toshev, C. Szegedy, [DeepPose: Human Pose Estimation via Deep Neural Networks](#), CVPR 2014.
- S.E. Wei, V. Ramakrishna, T. Kanade, Y. Sheikh, [Convolutional Pose Machines](#), CVPR 2016.
- A. Newell, K. Yang, J. Deng, [Stacked Hourglass Networks for Human Pose Estimation](#), ECCV 2016.
- Z. Cao, T. Simon, S.-E. Wei, Y. Sheikh, [Realtime Multi-Person 2D Pose Estimation using Parts Affinity Fields](#), CVPR 2017.
- B. Xiao, H. Wu, Y. Wei, [Simple Baselines for Human Pose Estimation and Tracking](#), ECCV 2018. ([Code](#))
- Z. Cao, G. Hidalgo Martinez, T. Simon, S.-E. Wei, [OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields](#), IEEE Trans. PAMI, 2019. ([Code](#))